.UBAfiuba
FACULTAD DE INGENIERÍA

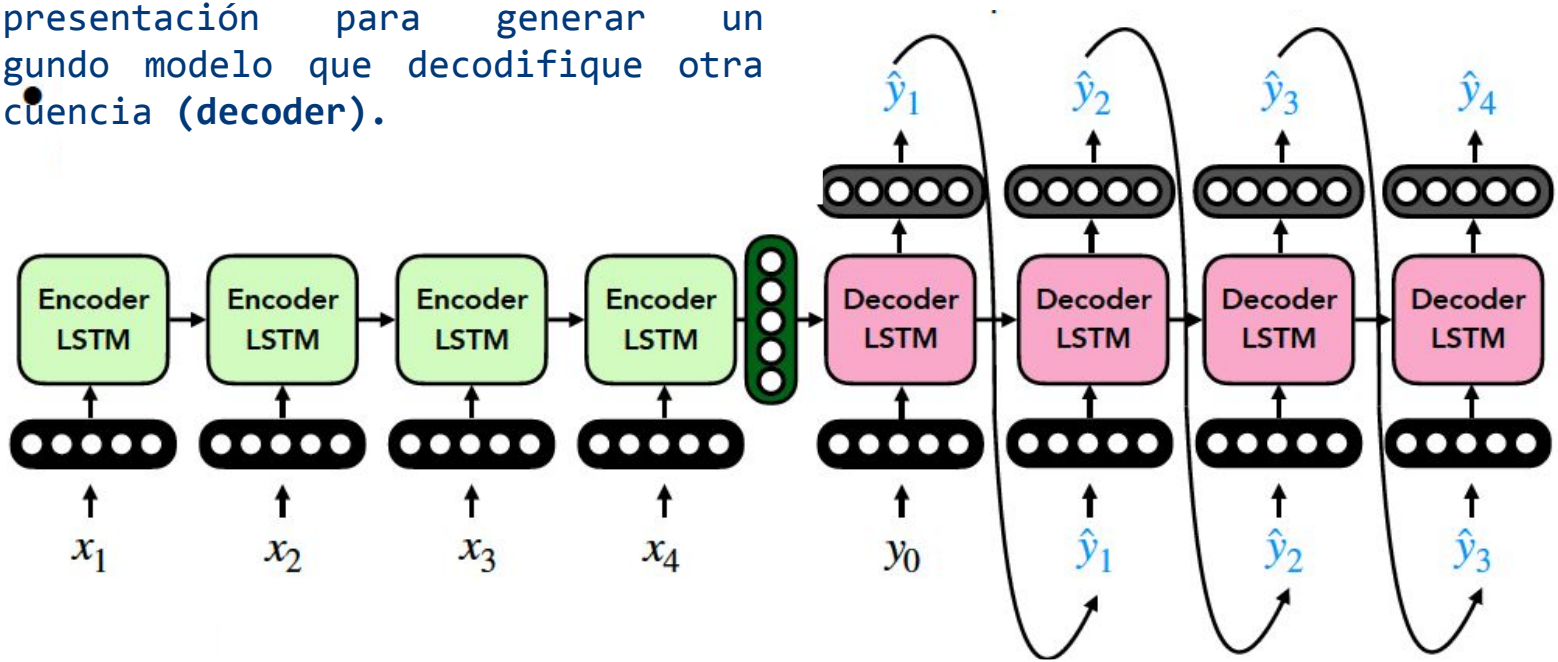
NLP

Celdas con Attention & Transformers

MSc. Fernando Silva
fernandosilva.clases@gmail.com

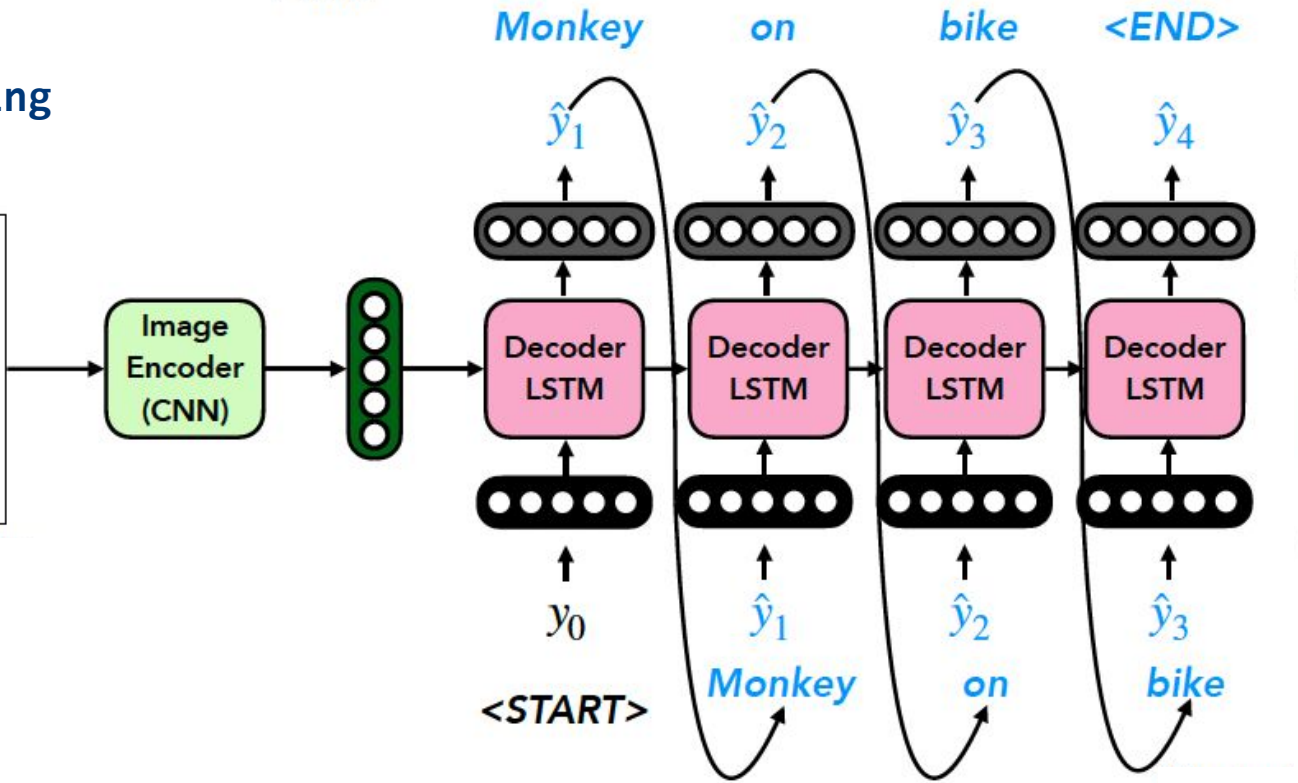
Seq2Seq

Codifica una secuencia completamente con un modelo **(encoder)** y utilizar su representación para generar un segundo modelo que decodifique otra secuencia **(decoder).**



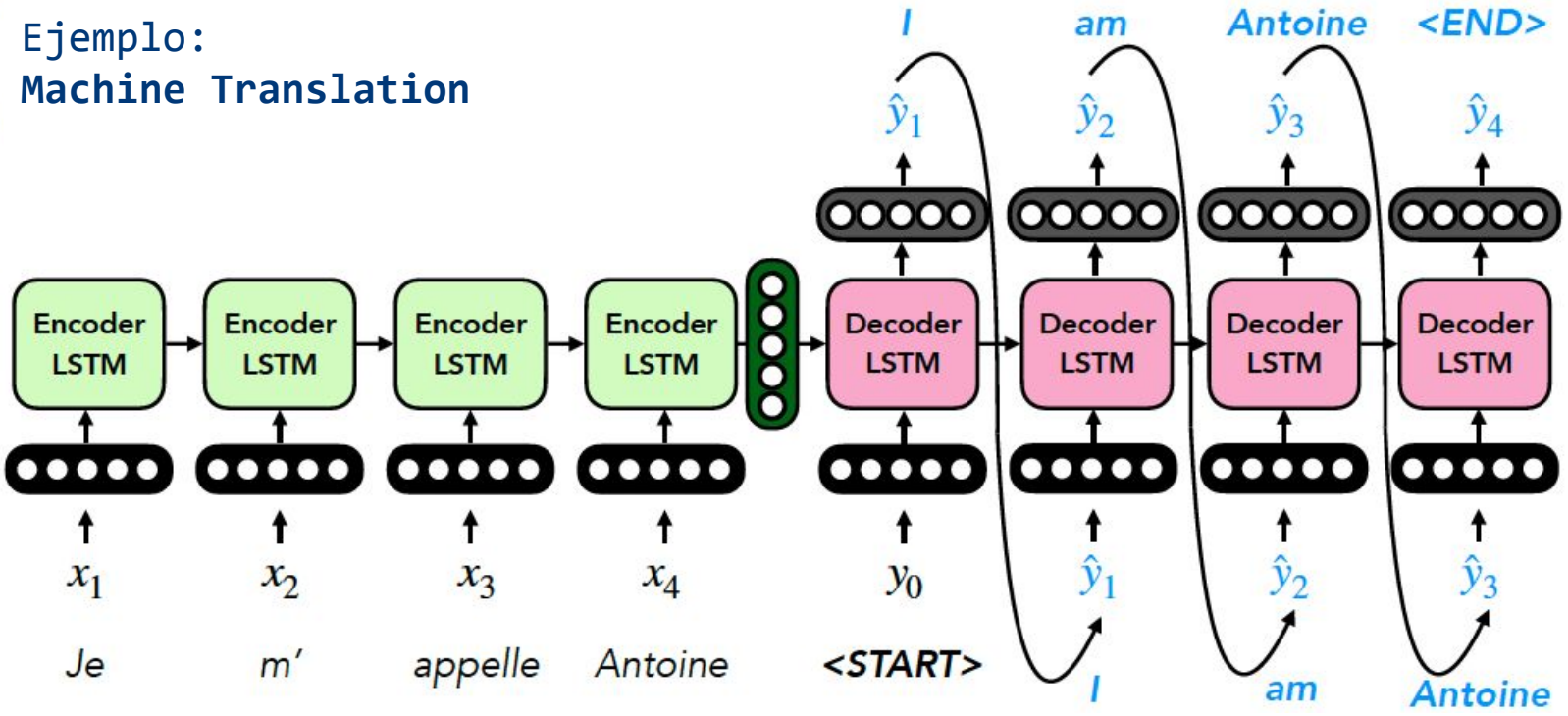
Seq2Seq

Ejemplo:
Image Captioning



Seq2Seq

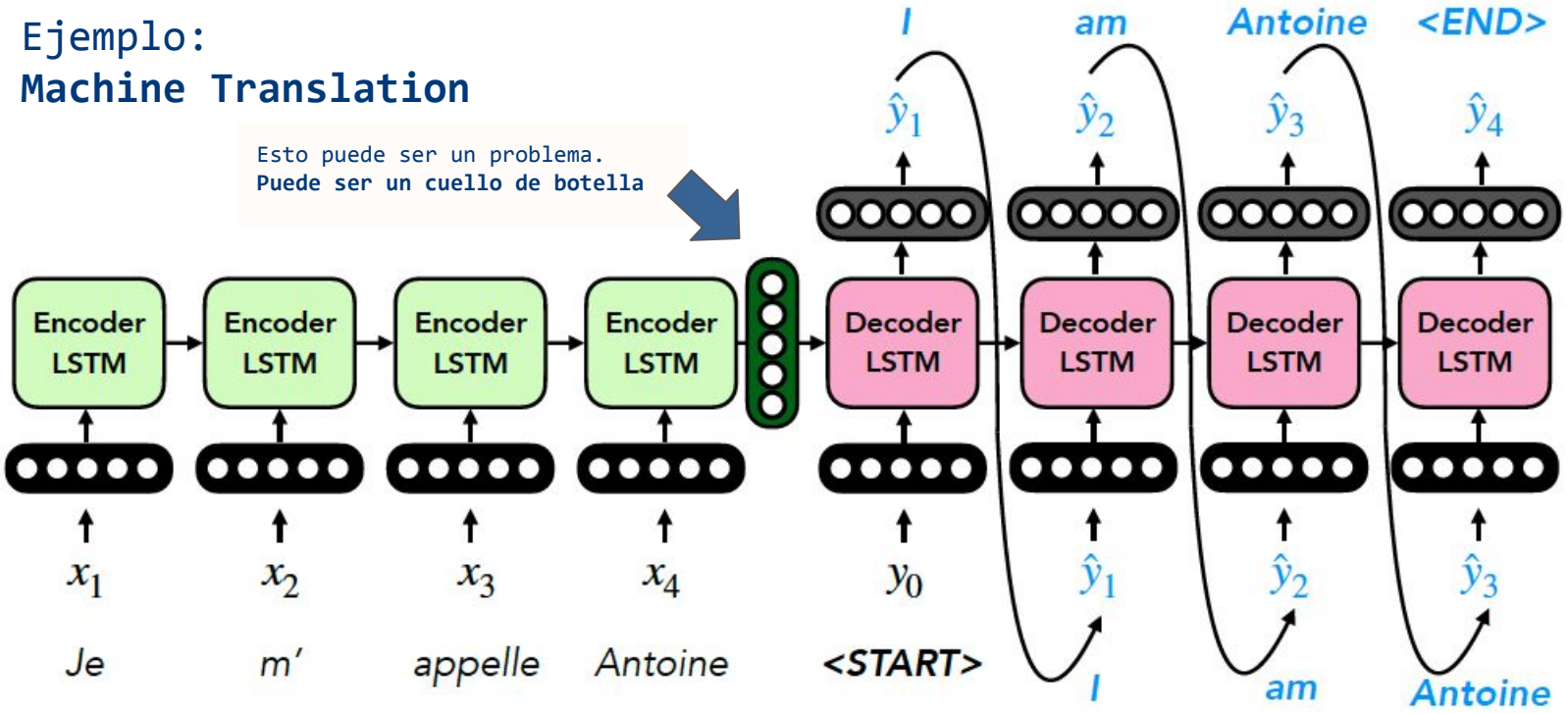
Ejemplo:
Machine Translation



Seq2Seq

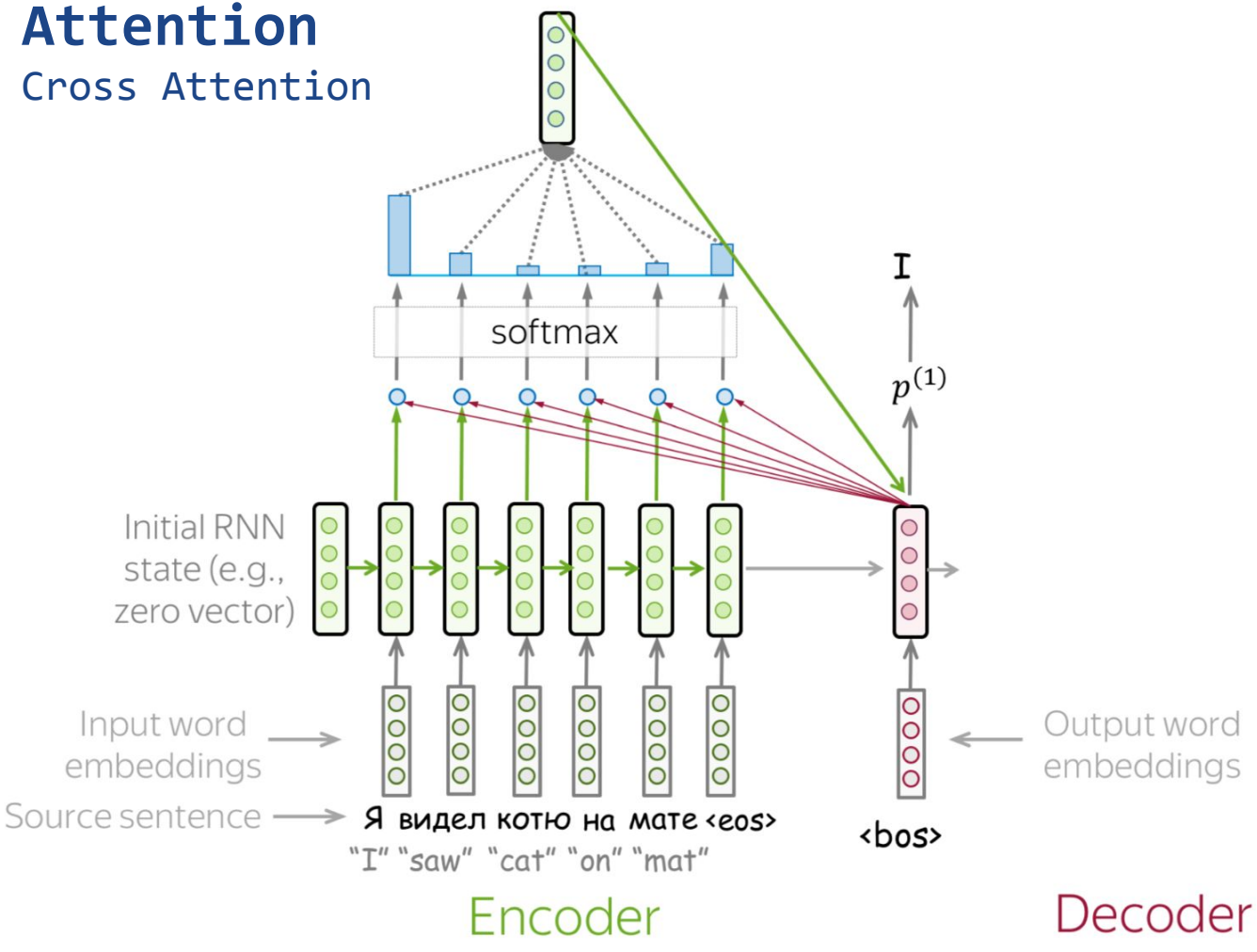
Ejemplo:
Machine Translation

Esto puede ser un problema.
Puede ser un cuello de botella



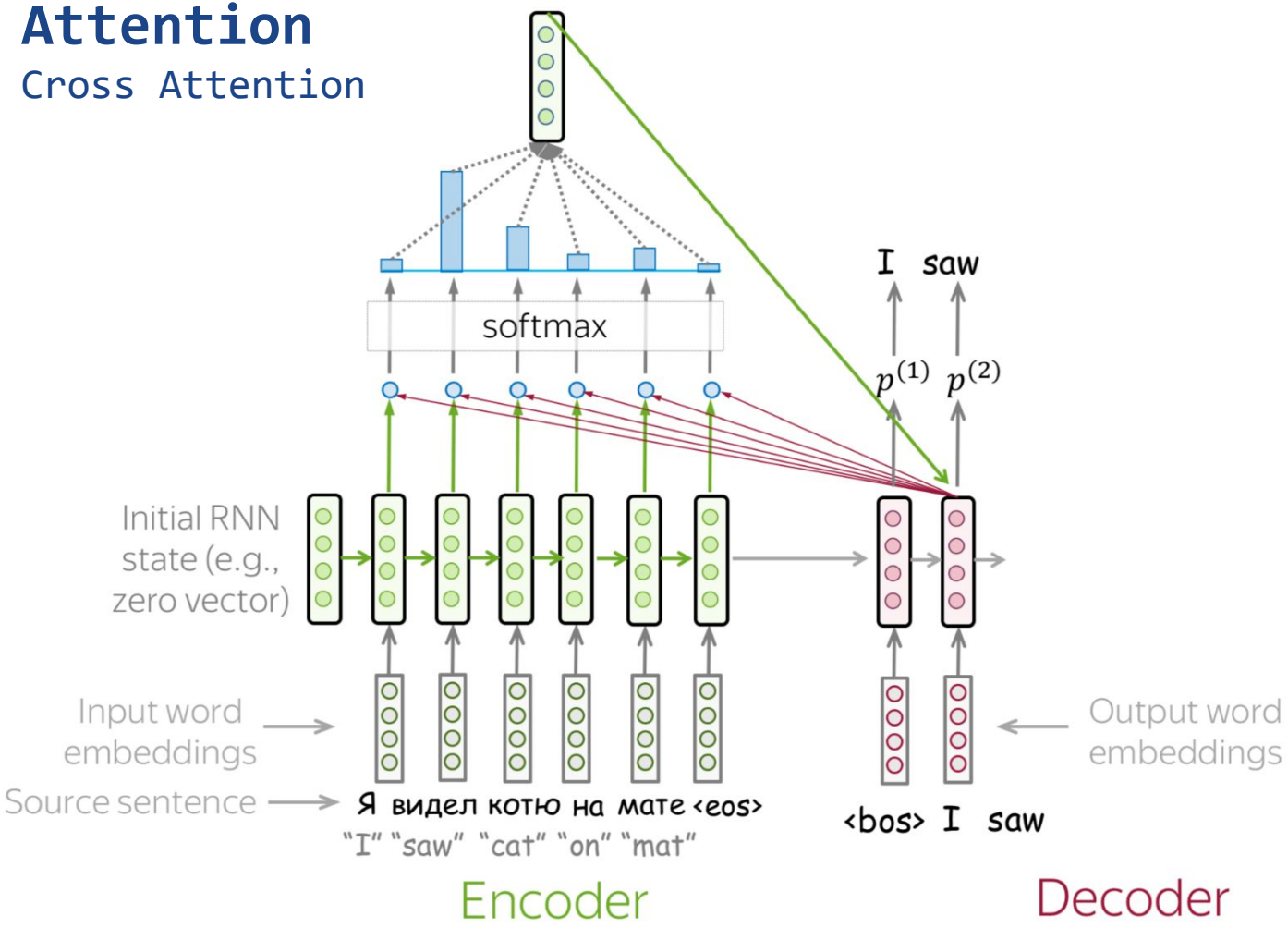
Attention

Cross Attention



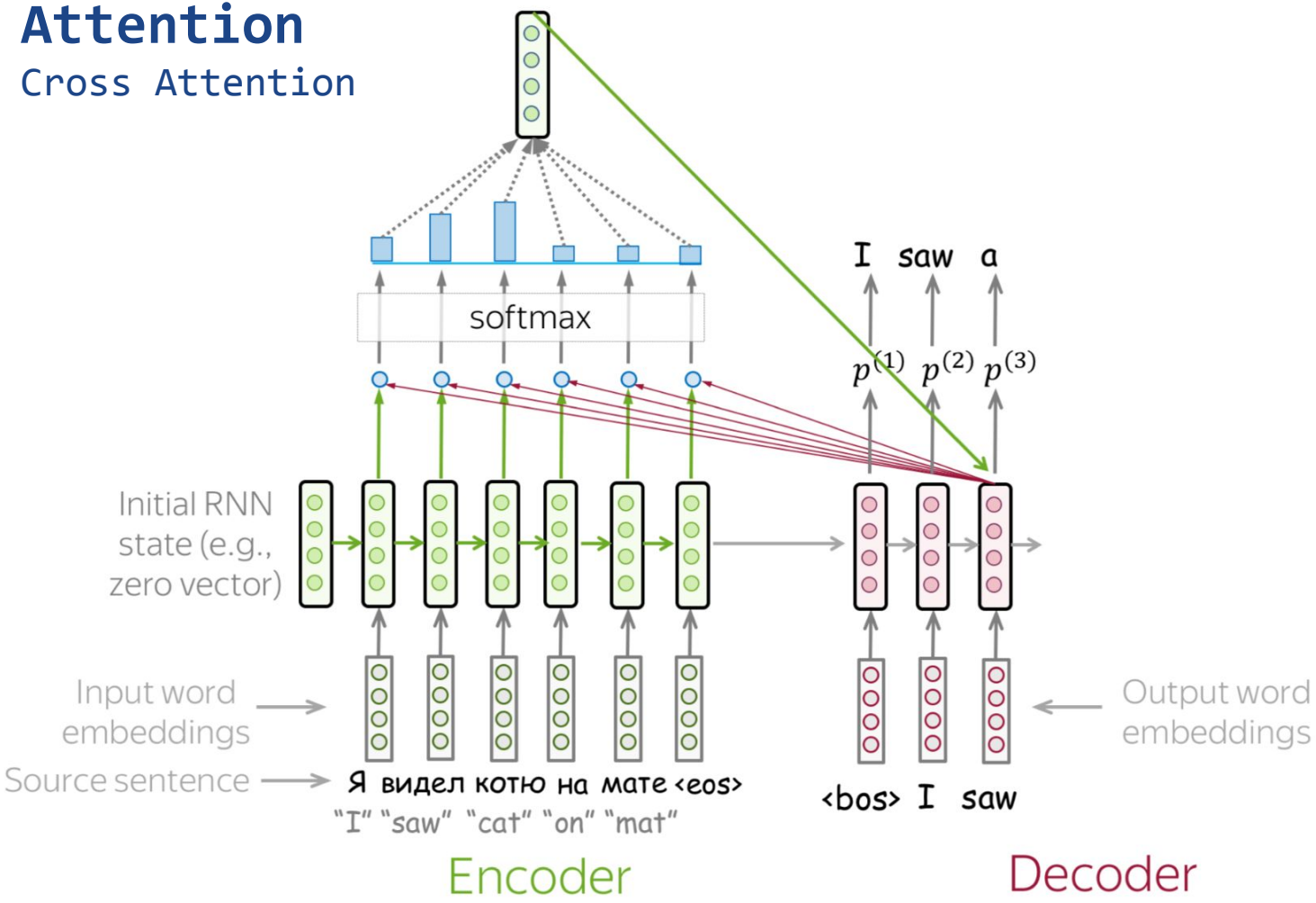
Attention

Cross Attention



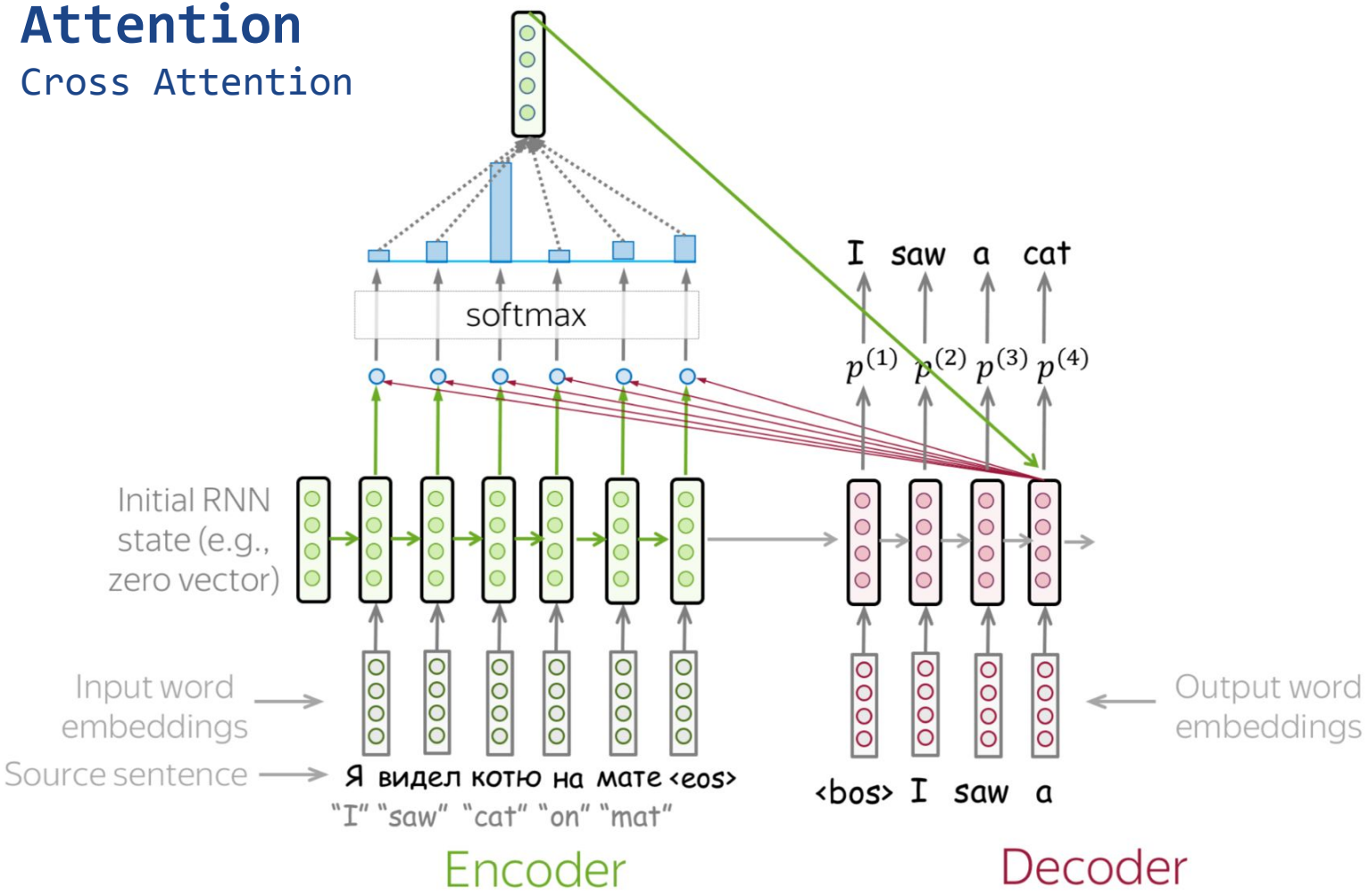
Attention

Cross Attention



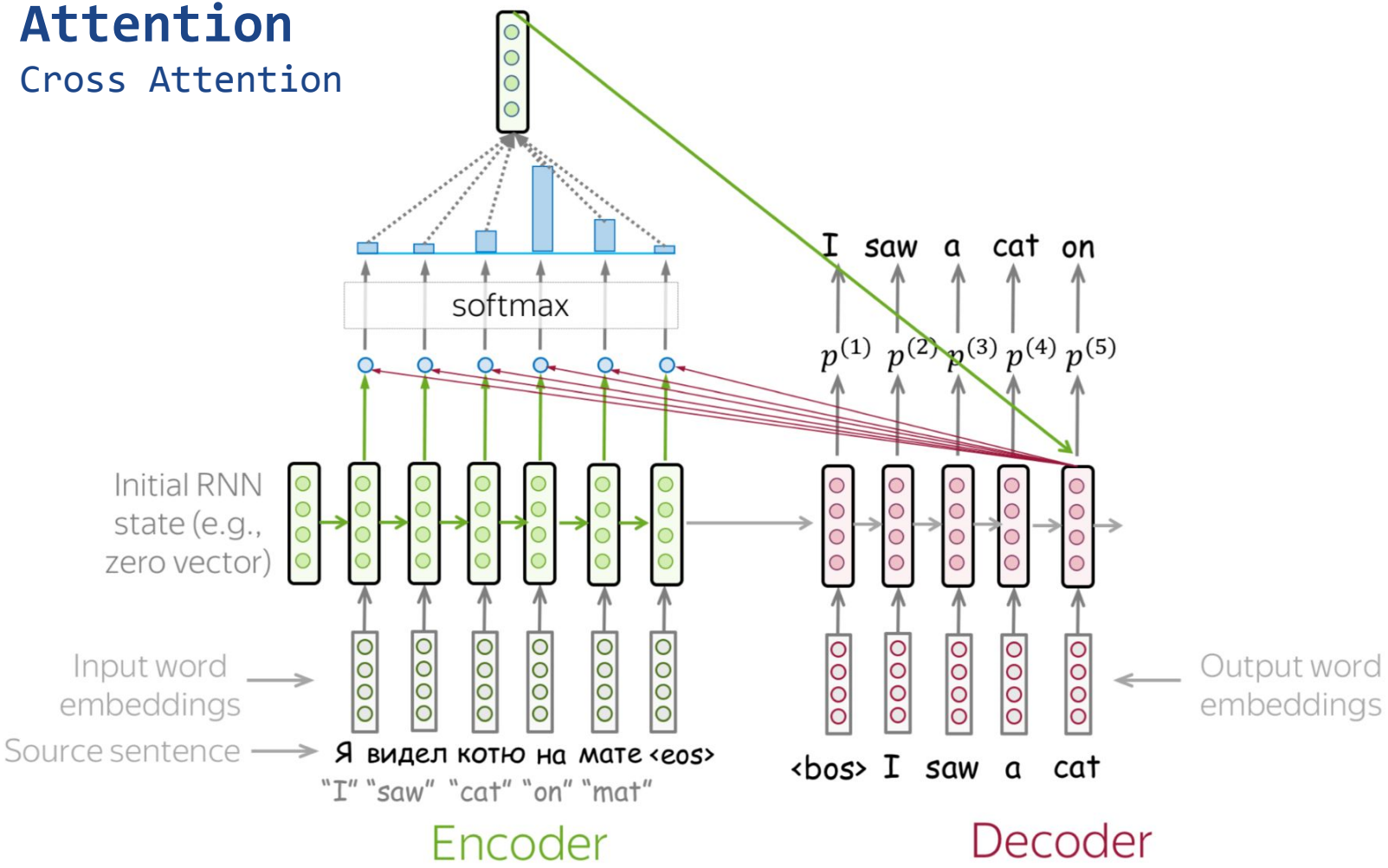
Attention

Cross Attention



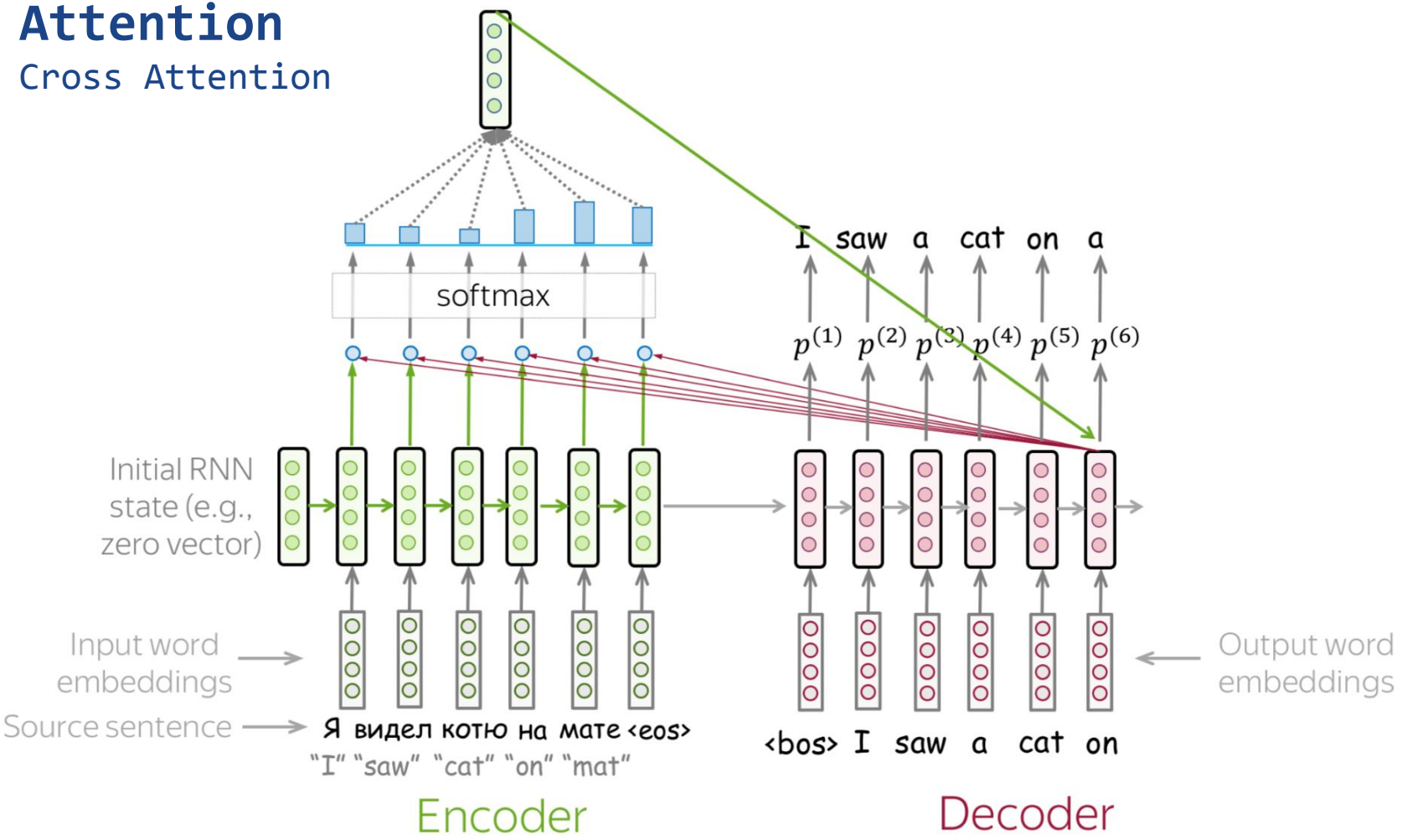
Attention

Cross Attention



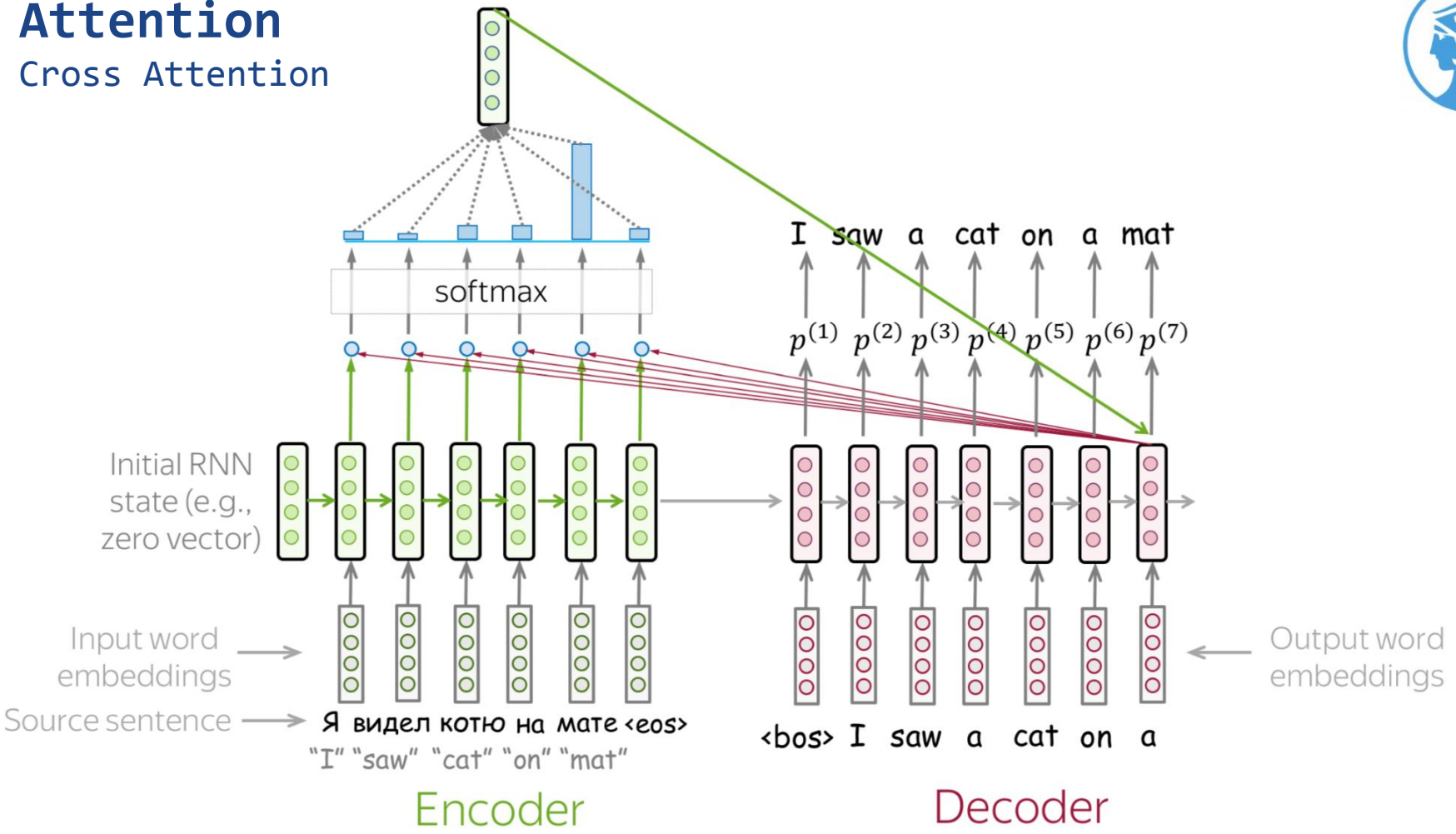
Attention

Cross Attention



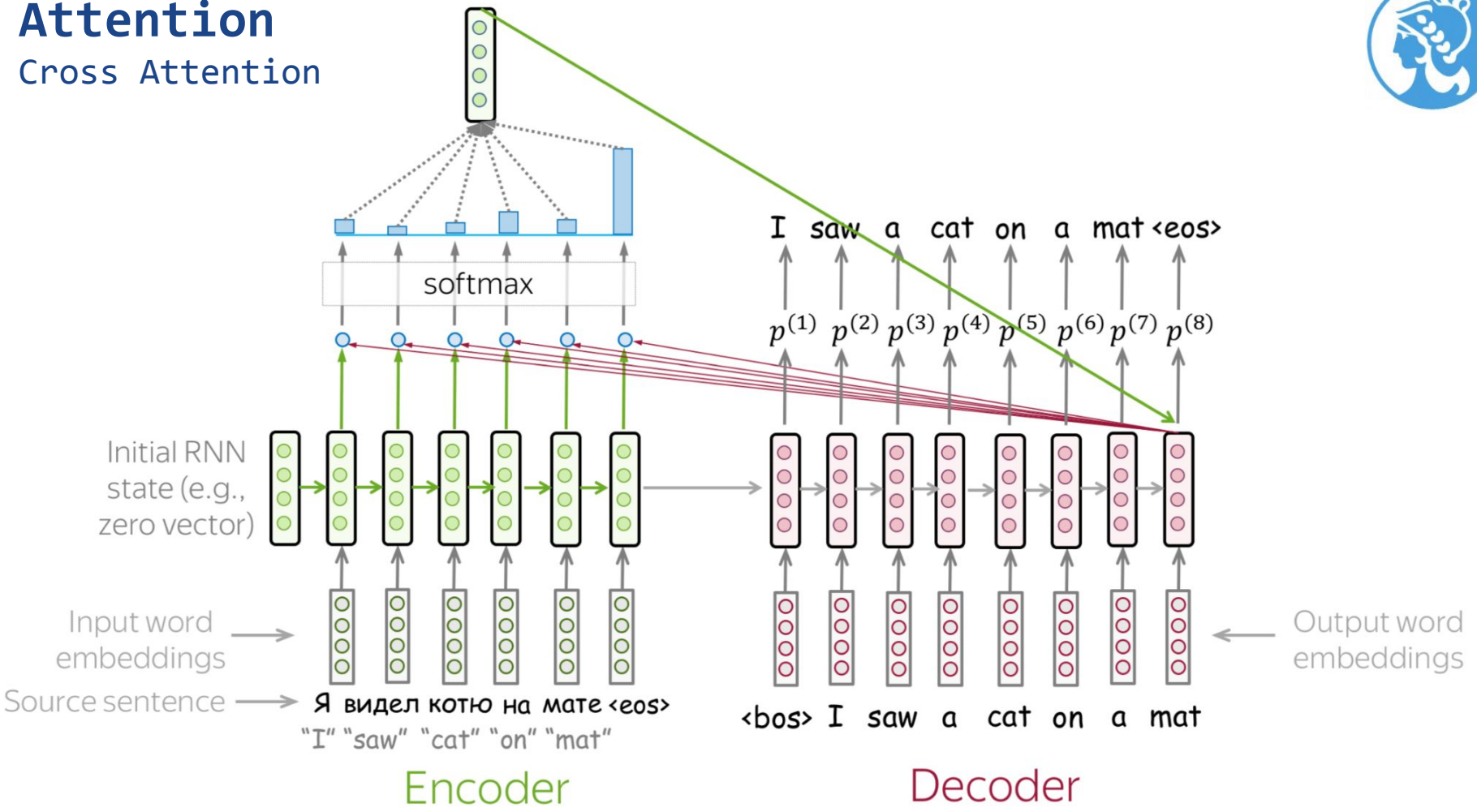
Attention

Cross Attention



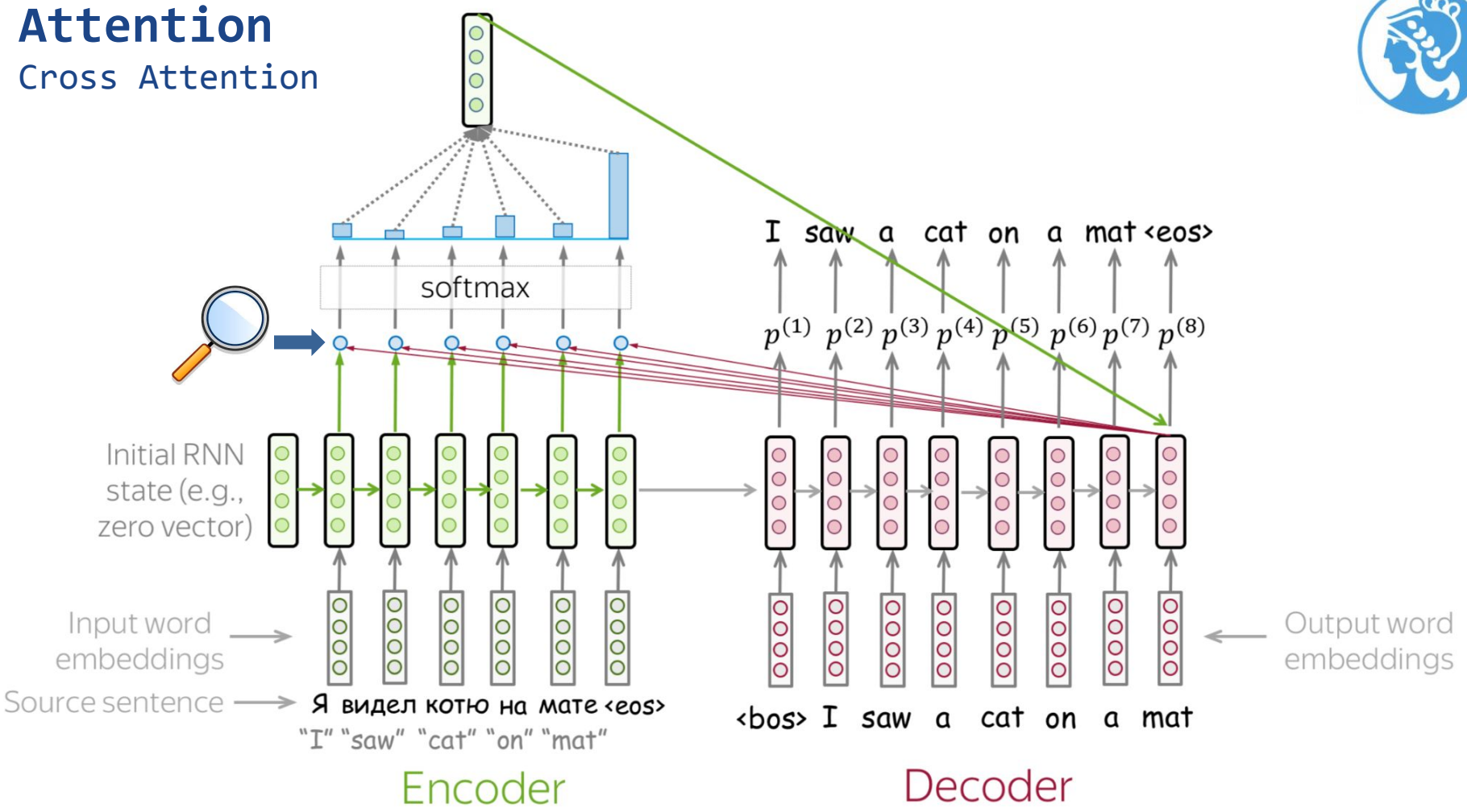
Attention

Cross Attention



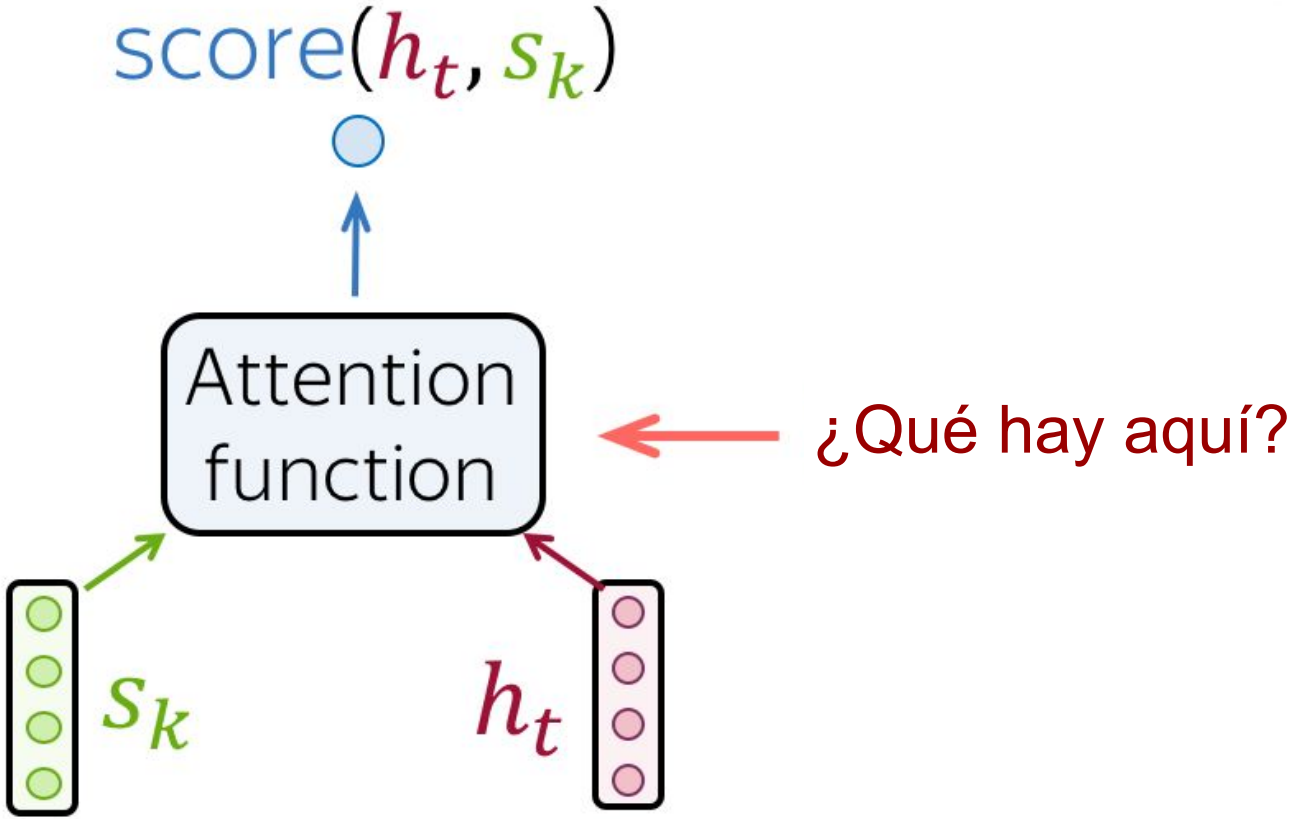
Attention

Cross Attention



Attention

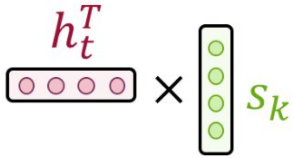
Cross Attention



Attention

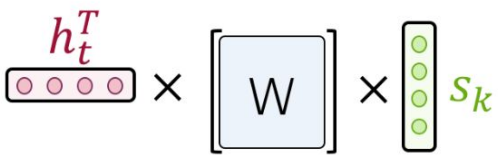
Cross Attention

Dot-product



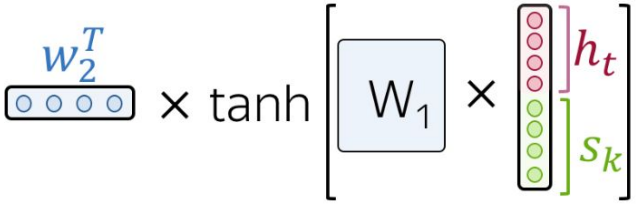
$$\text{score}(h_t, s_k) = h_t^T \; s_k$$

Bilinear



$$\text{score}(h_t, s_k) = h_t^T \; W s_k$$

Multi-Layer Perceptron



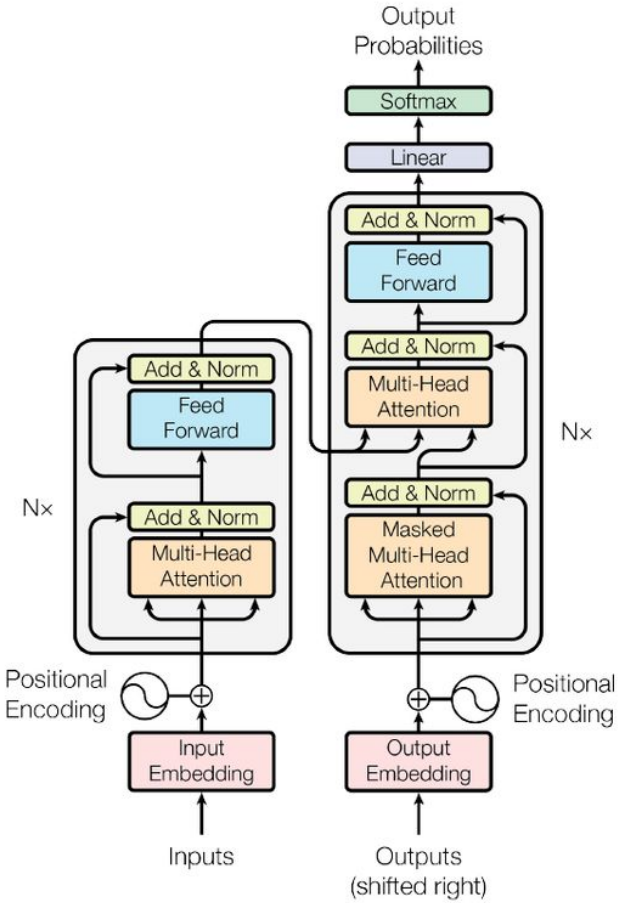
$$\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1[h_t, s_k])$$

Attention

Cross Attention

	Seq2seq without attention	Seq2seq with attention	Transformer
processing within encoder	RNN/CNN	RNN/CNN	attention
processing within decoder	RNN/CNN	RNN/CNN	attention
decoder-encoder interaction	static fixed-sized vector	attention	attention

Transformer



Transformer
Self-Attention

- Self-attention es el mecanismo mediante el cual **cada token de una secuencia interactúa con los demás tokens.**

- **Cada token "atiende" a otros tokens de la oración,** asigna distintos niveles de importancia a cada uno, y utiliza esa información para **construir una nueva representación contextualizada de sí mismo.**

Transformer
Self-Attention

En self-attention, cada token se proyecta en tres representaciones distintas:

- Query (Q): representa lo que el token está buscando.
- Key (K): representa qué tipo de información ofrece ese token.
- Value (V): contiene la información que será utilizada para construir la nueva representación.

La similitud entre Query y Key determina qué tan relevante es un token para otro, y esos pesos se usan para combinar los Values.

Transformer
Self-Attention

Each vector receives three representations ("roles")

$W_Q \times = =$ Query: vector **from** which the attention is looking

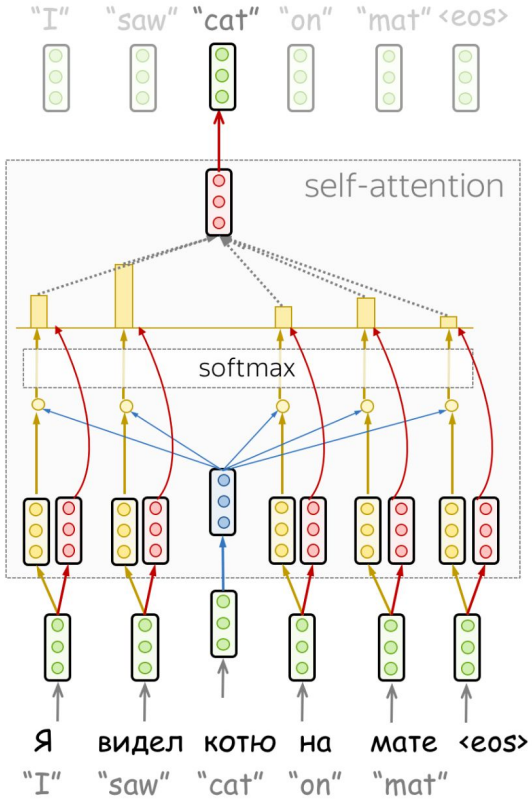
"Hey there, do you have this information?"

$W_K \times = =$ Key: vector **at** which the query looks to compute weights

"Hi, I have this information – give me a large weight!"

$W_V \times = =$ Value: their weighted sum is attention output

"Here's the information I have!"



Transformer
Self-Attention

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

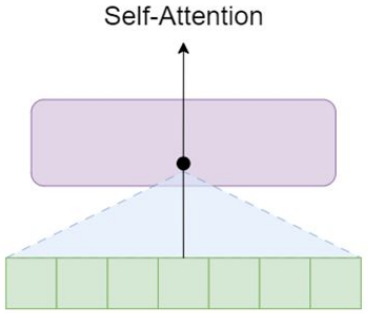
- La dimensión de los vectores Key (K) (y también de los Query (Q))
- Se usa para escalar el producto QK^T y evitar valores demasiado grandes.

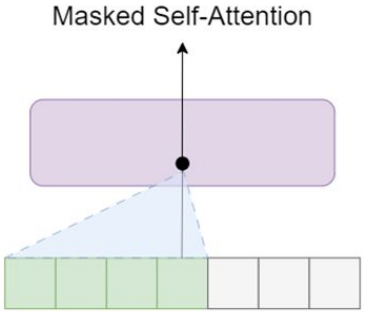
Transformer
Masked Self-Attention

- El **encoder** recibe todos los tokens al mismo tiempo, por lo que cada token puede atender a cualquier otro token de la secuencia mediante **self-attention**.

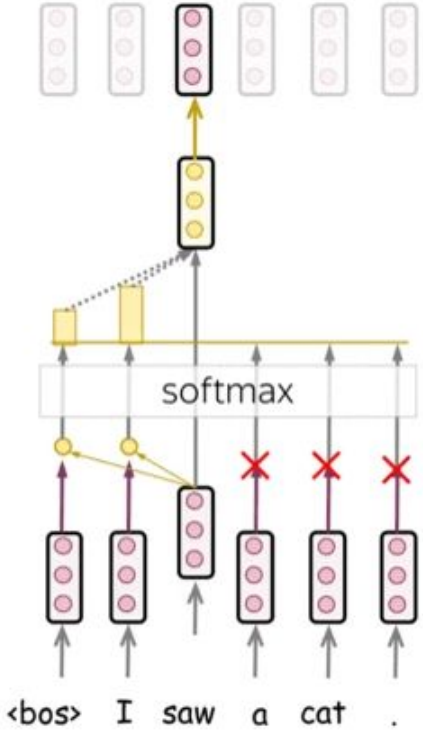
- En cambio, el **decoder** genera la secuencia token por token. En cada paso, solo tiene acceso a los tokens generados hasta ese momento.

- Para evitar que el modelo “vea el futuro”, se utiliza **masked self-attention**, donde las posiciones futuras se enmascaran, impidiendo que influyan en la predicción actual.





Transformer
Masked Self-Attention

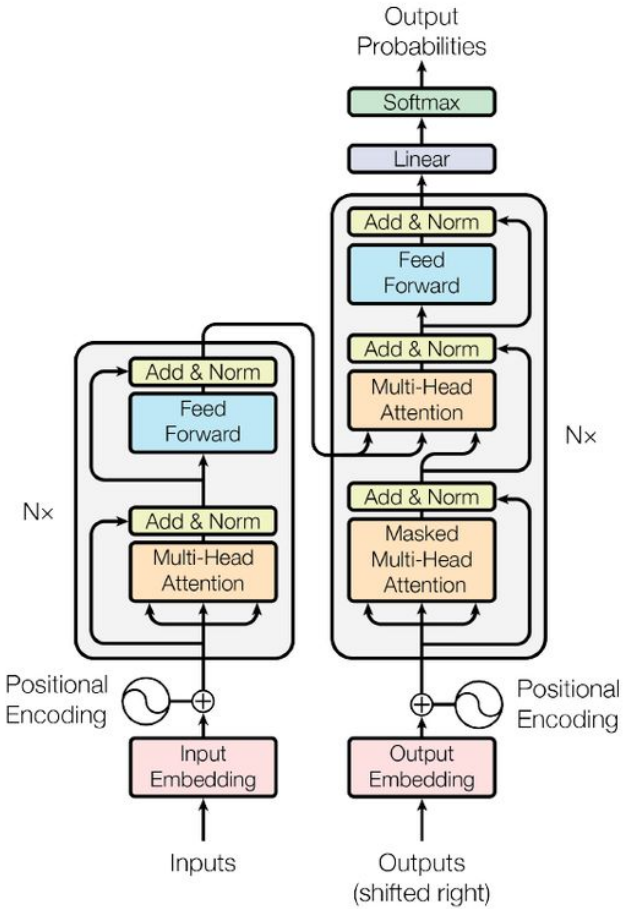


update token representation

gather context

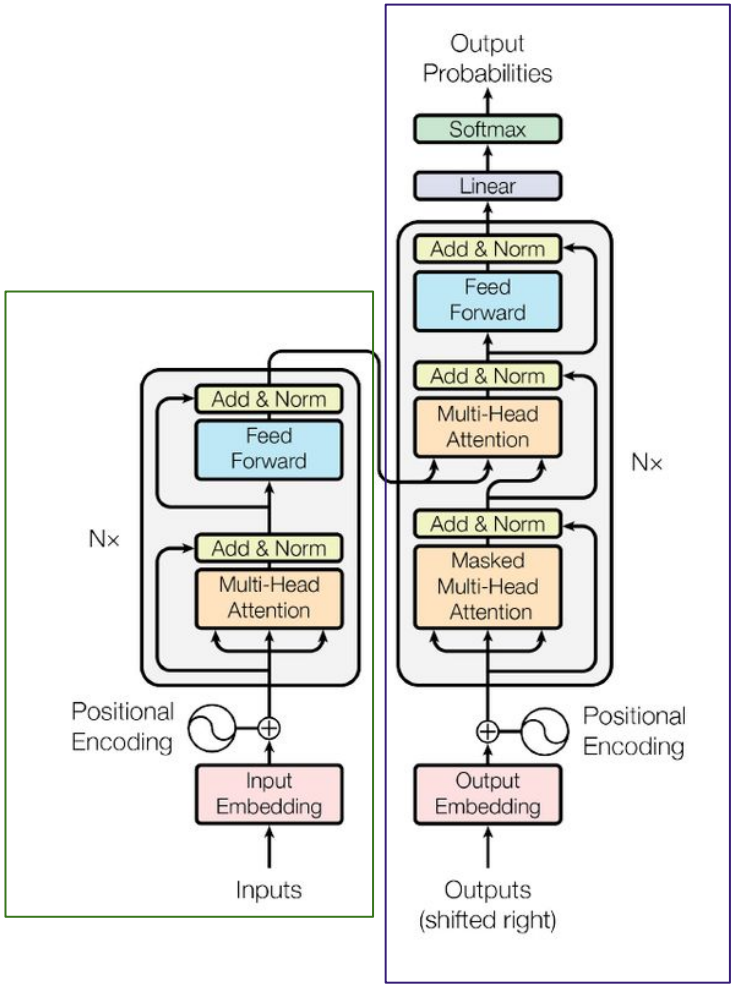
"look" at the previous tokens (future tokens are masked out)

Transformer



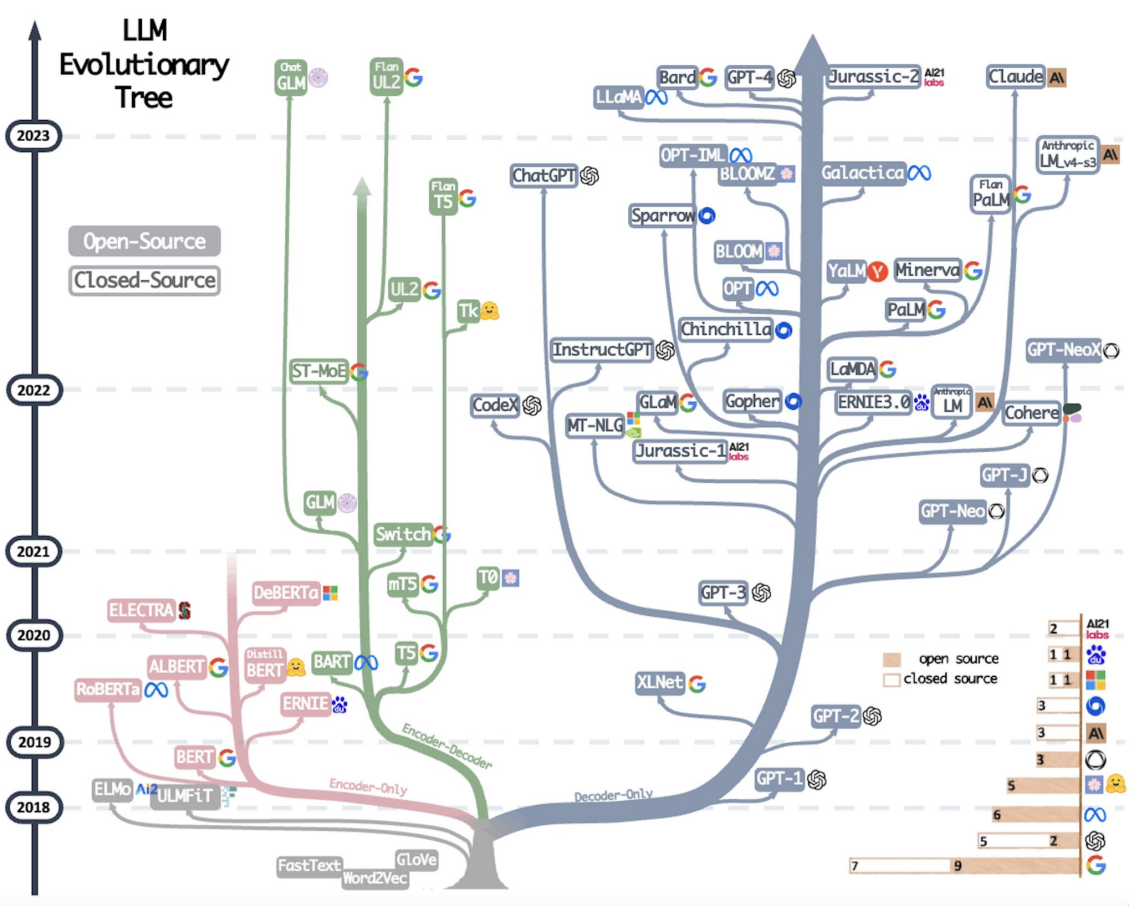
Transformer

Encoder
Representación

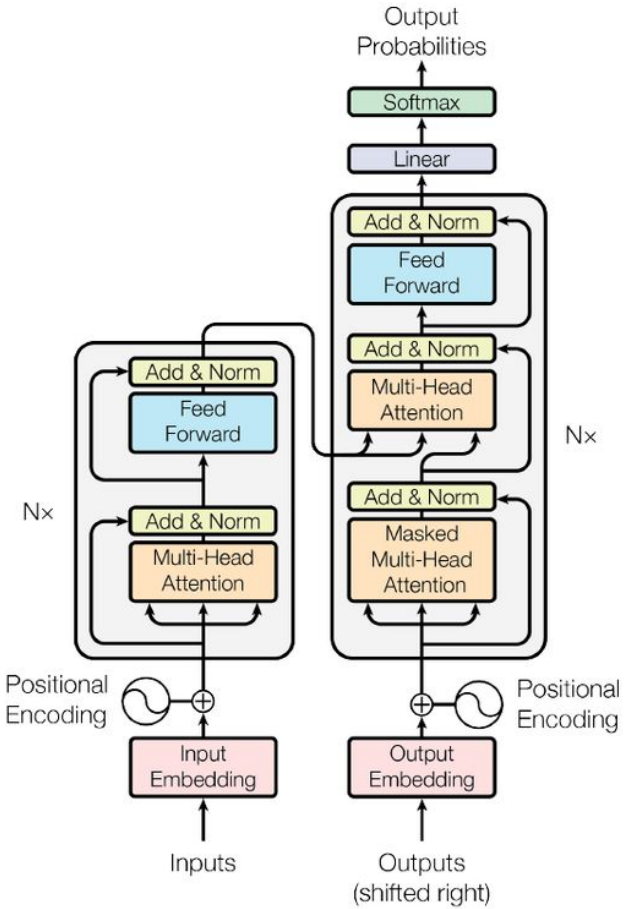


Decoder
Generación

Transformer



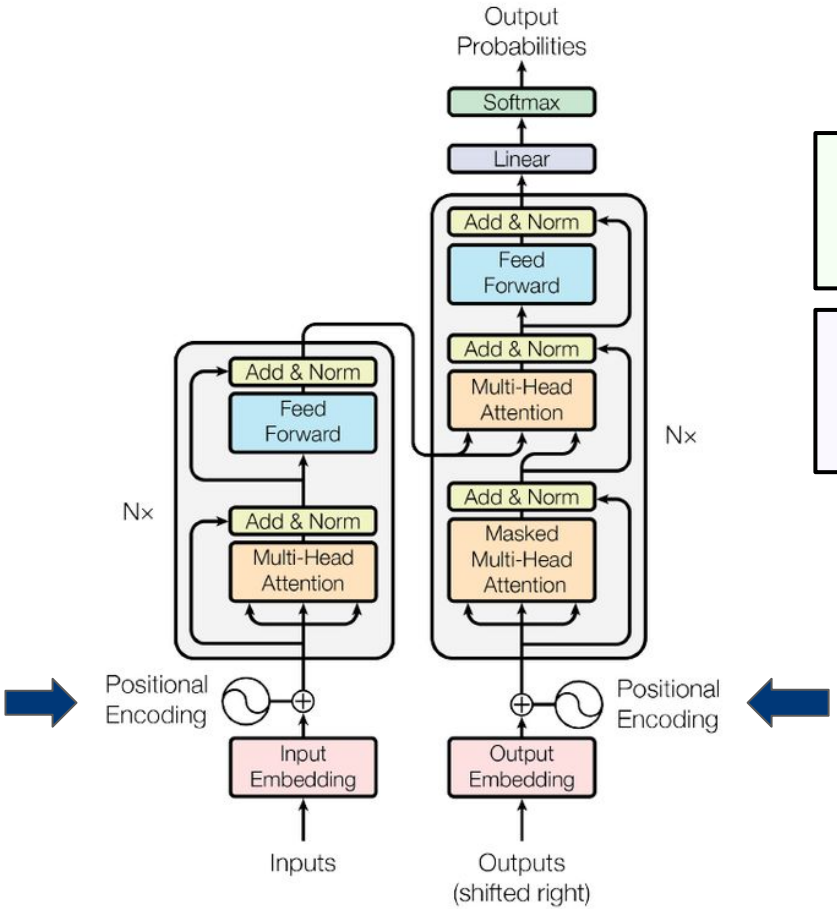
Transformer



Encoder
Representación

Decoder
Generación

Transformer

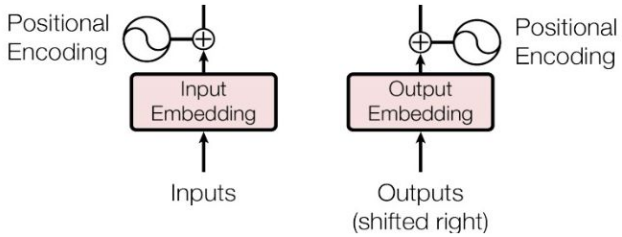


Encoder
Representación

Decoder
Generación

Transformer
Positional Encoding

- El mecanismo de self-attention no tiene información sobre el orden de las palabras por sí solo.
- A diferencia de modelos como RNN o LSTM, que procesan las secuencias paso a paso, el self-attention procesa todos los tokens en paralelo.
- Esto significa que, sin información adicional, el modelo no puede distinguir la posición de cada palabra ni saber cuál viene antes o después.



- La idea es **añadir un vector adicional a cada embedding** que representa la posición de la palabra en la secuencia.

Transformer
Positional Encoding

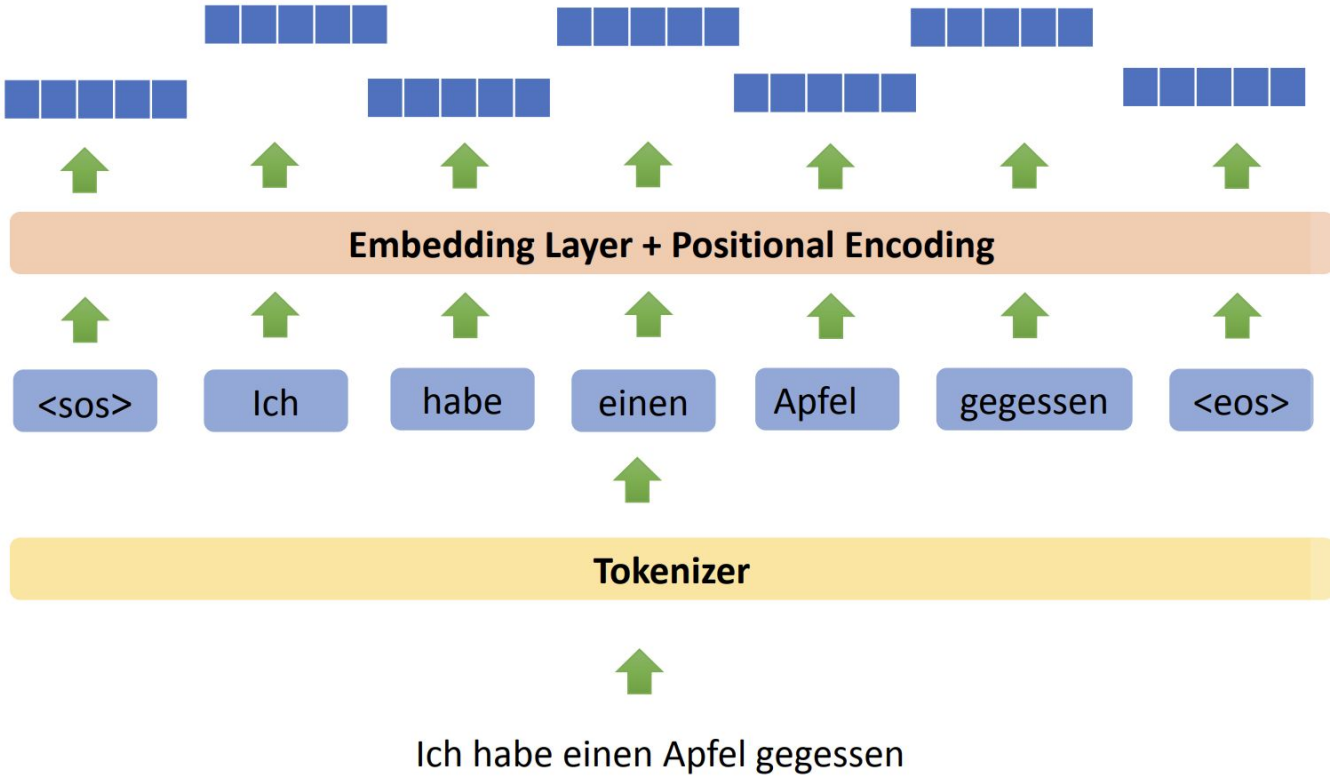
Ejemplo:
- "dog" en la posición 1 → embedding(dog) + encoding(pos=1)
- "dog" en la posición 5 → embedding(dog) + encoding(pos=5)

Formas de representar la posición:

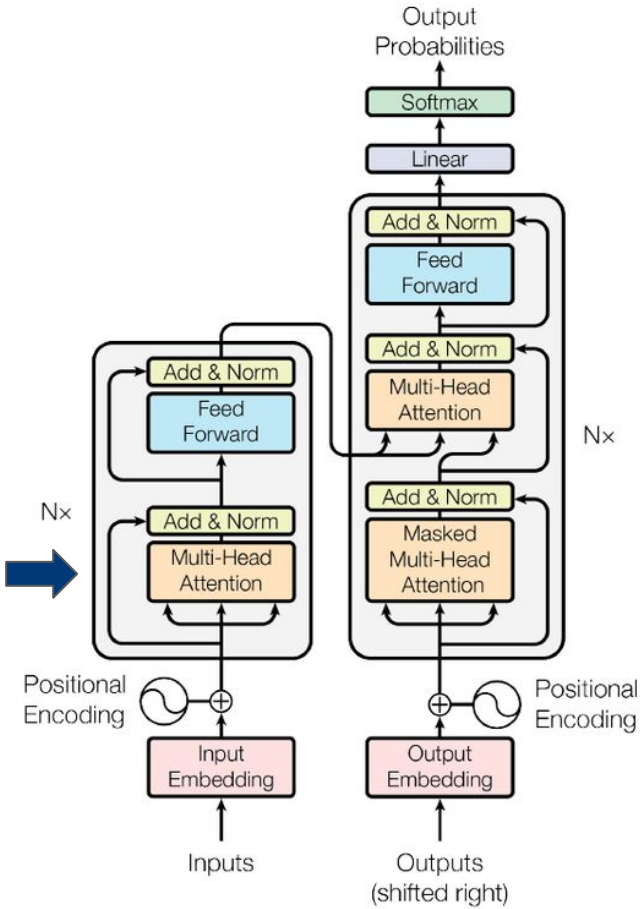
- **Predefinida (función matemática):** Los transformers originales (Vaswani et al., 2017) usan una f**unción sinusoidal.** Esto hace que los vectores de posiciones cercanas estén relacionados (capturan distancias).

- **Aprendida (trainable):** Algunos modelos aprenden directamente los vectores de posición como parámetros, igual que los embeddings de palabras.

Transformer
Positional Encoding



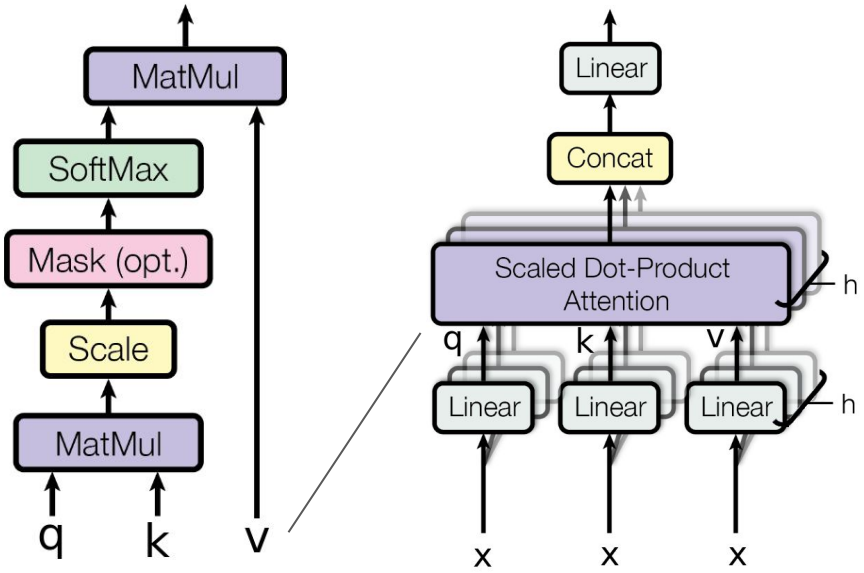
Transformer



Encoder
Representación

Decoder
Generación

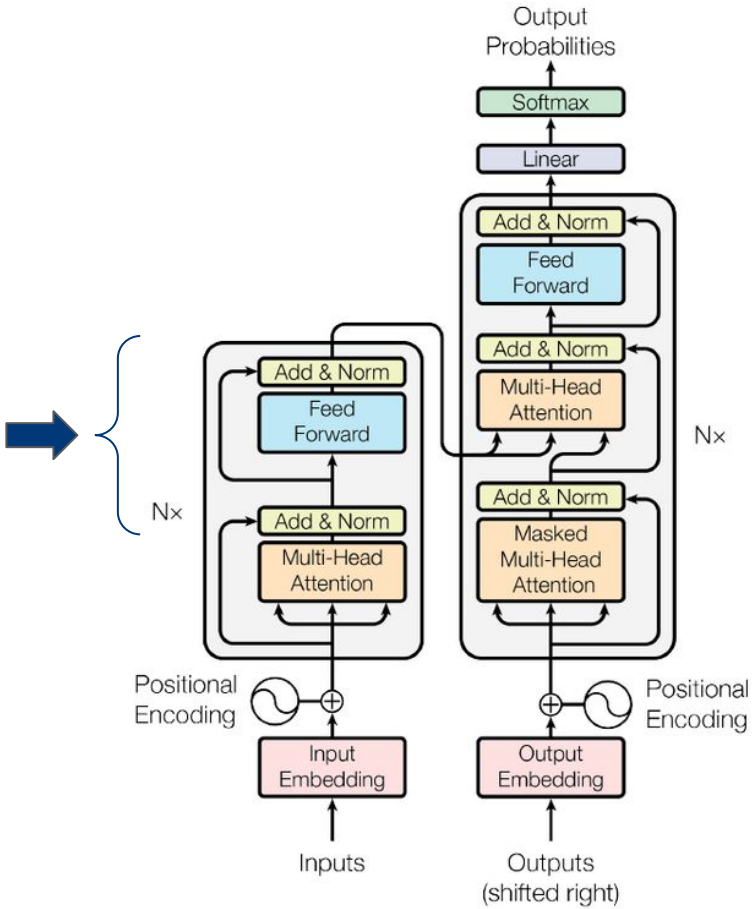
Transformer
Multi-Head Attention



Multi-head es similar a tener varios filtros convolucionales en redes convolucionales.

Cada módulo de atención puede aprender a “atender” diferentes relaciones entre los elementos de la secuencia.

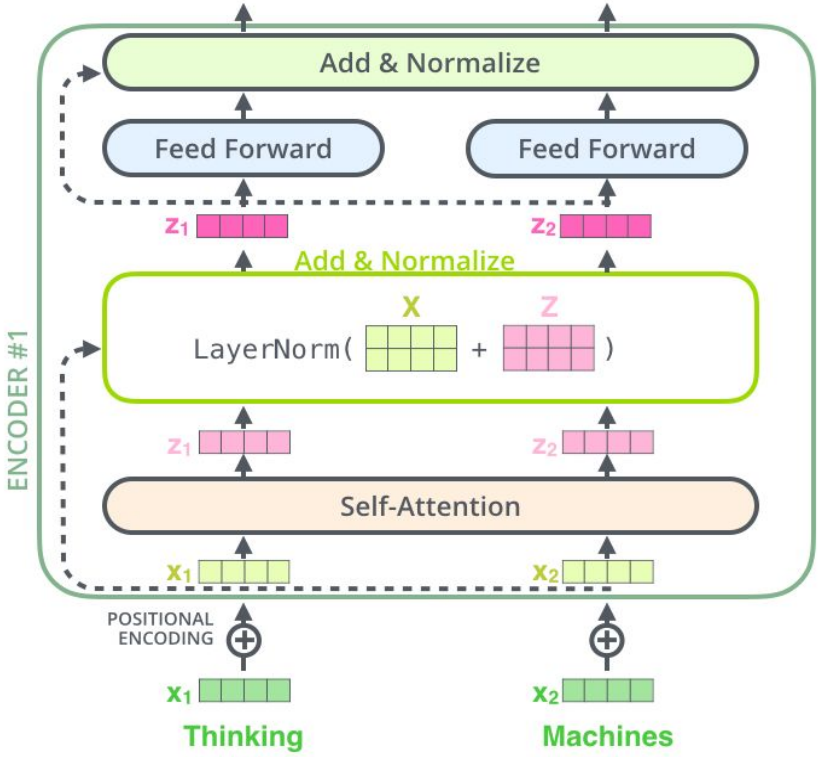
Transformer



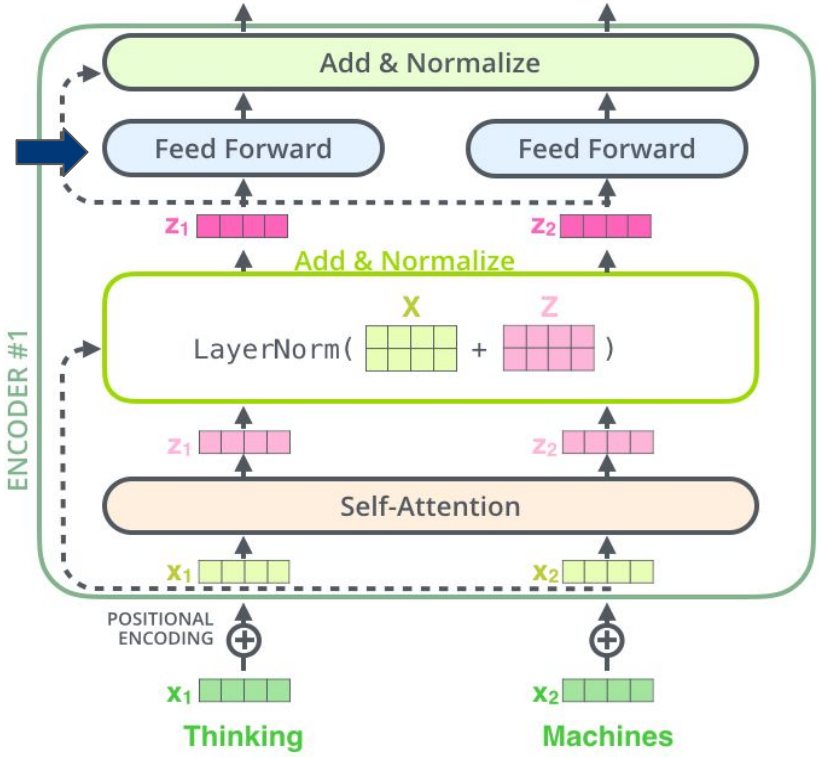
Encoder
Representación

Decoder
Generación

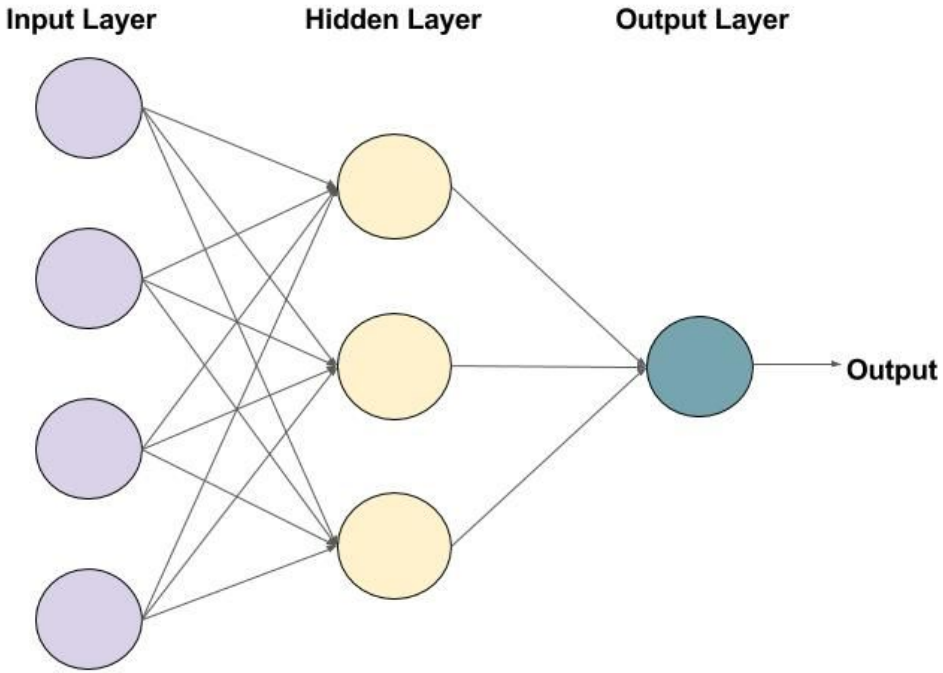
Transformer
Add & Norm



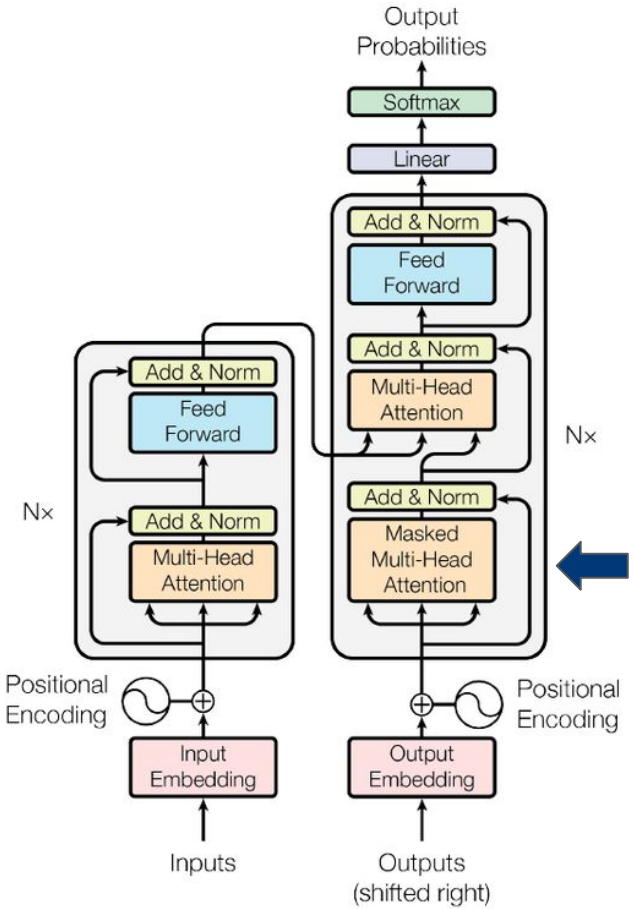
Transformer
Add & Norm



Transformer
Feed Forward



Transformer



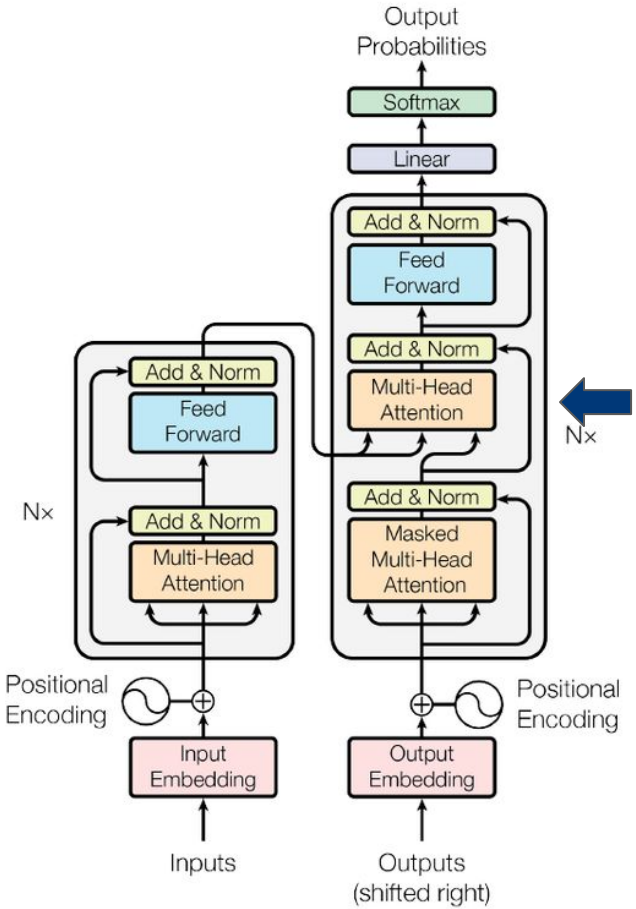
Encoder
Representación

Decoder
Generación

Transformer

Masked Multi-Head Attention

Transformer



Encoder
Representación

Decoder
Generación

Transformer

Encoder - Decoder

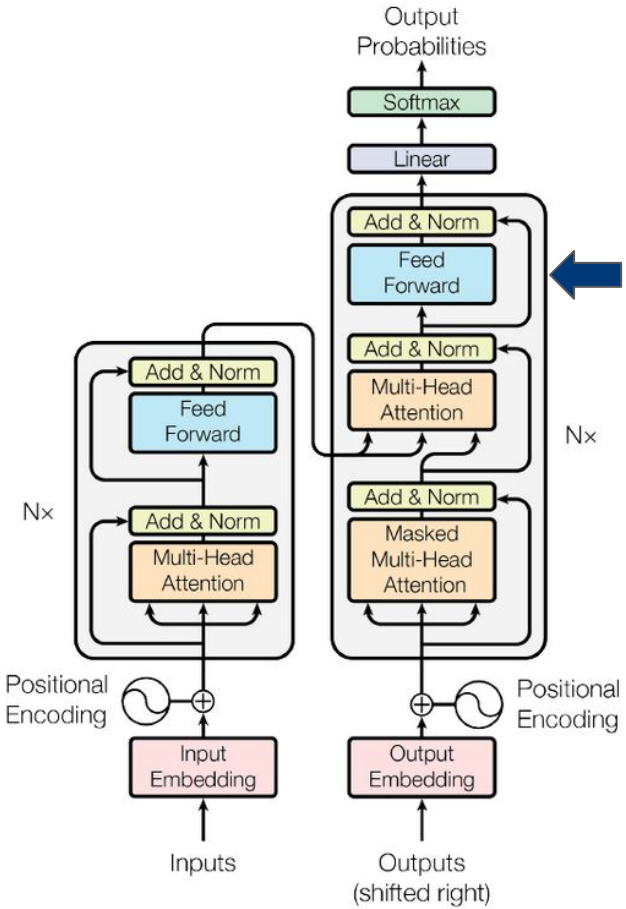
Encoder

Keys from **Encoder Outputs**
Values from **Encoder Outputs**

Decoder

Queries from **Decoder Inputs**

Transformer

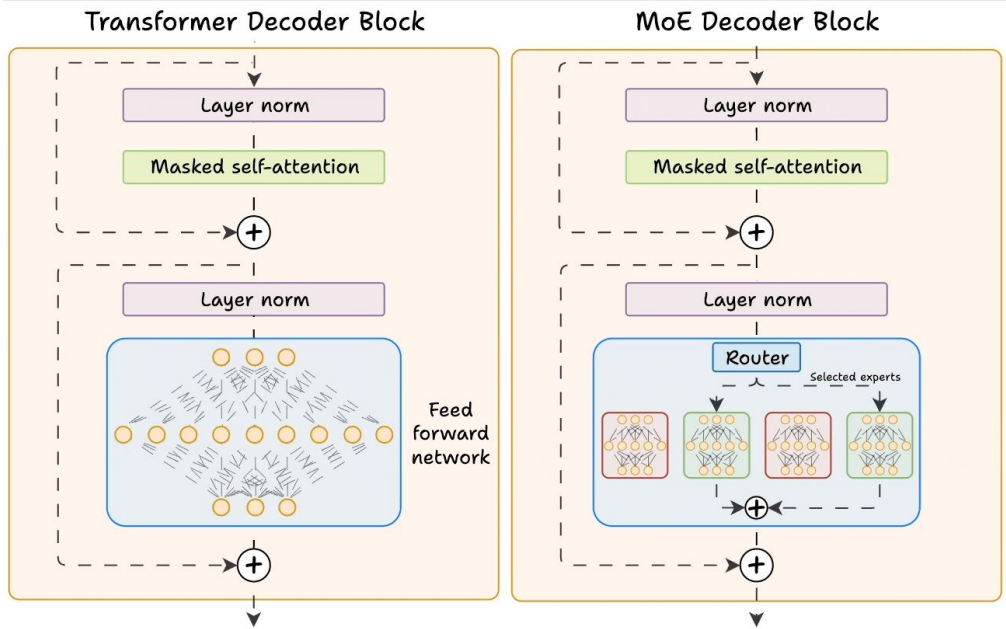


Encoder
Representación

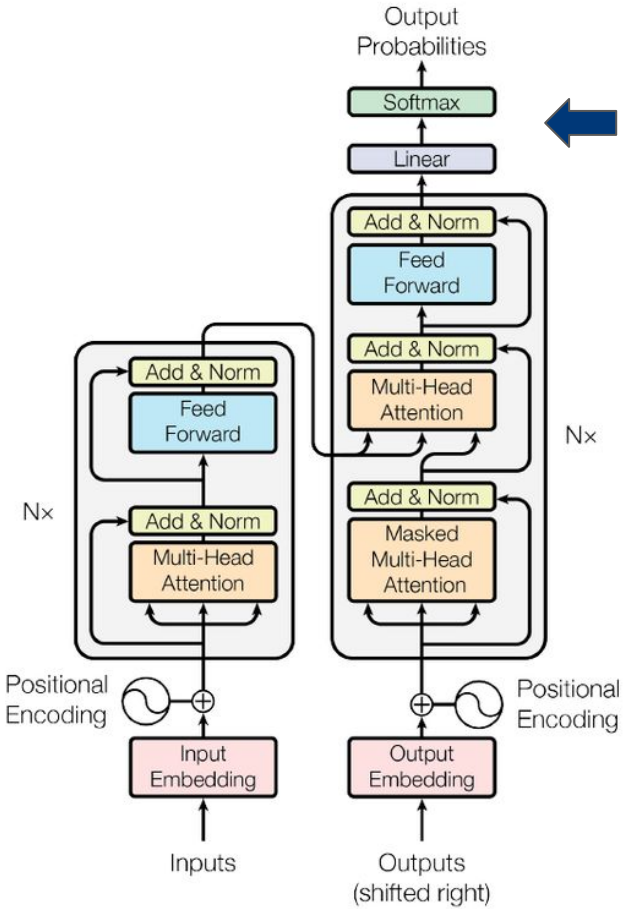
Decoder
Generación

Transformer
Feed Forward, Mixture of Experts

Un Mixture of Experts es una variante del transformer donde el **bloque feedforward se reemplaza por múltiples redes especializadas (expertos)**, y un mecanismo de enrutamiento decide cuáles usar para cada token.



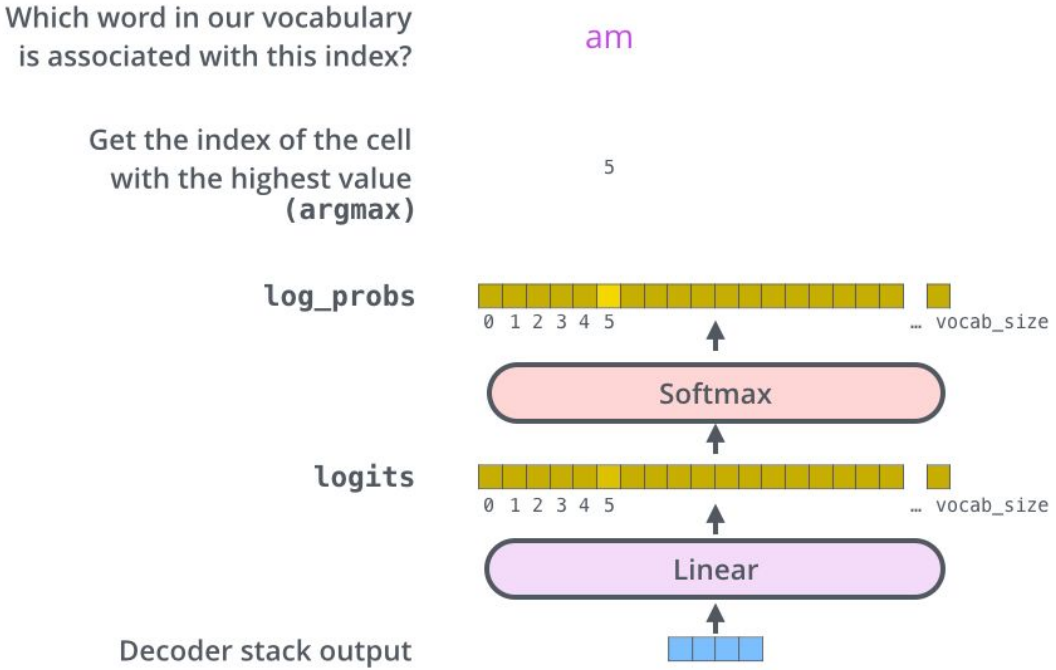
Transformer



Encoder
Representación

Decoder
Generación

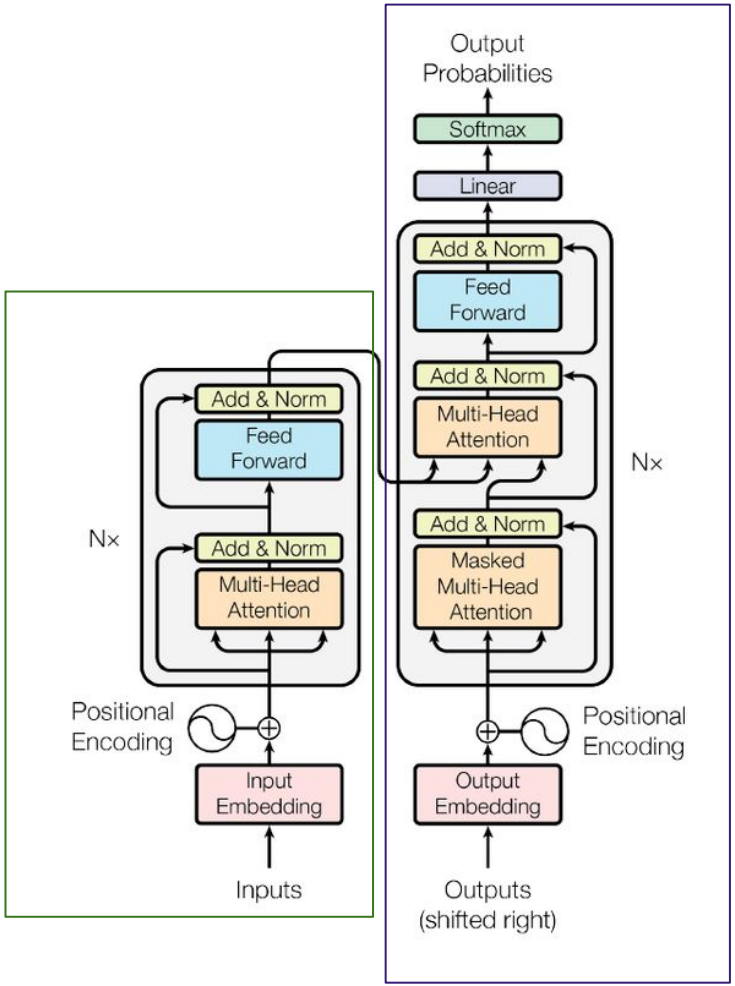
Transformer
Softmax



Transformer

BERT

Encoder
Representación



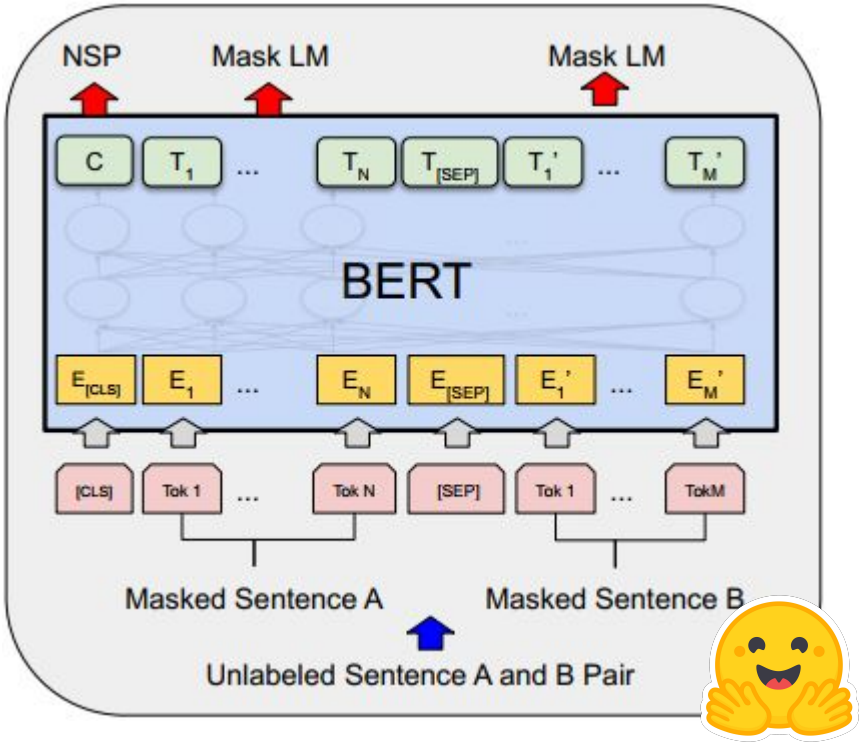
GPT

Decoder
Generación

BERT (2019)

LINK LINK LINK

“Pre-training of Deep Bidirectional Transformers”



Creado por Hugging Face/Google

Su arquitectura se basa en stack de layers de transformers (encoders)

NSP: Se entrenó observando dos secuencias de entrada, su misión era determinar si la segunda secuencia estaba relacionada o no con la primera.

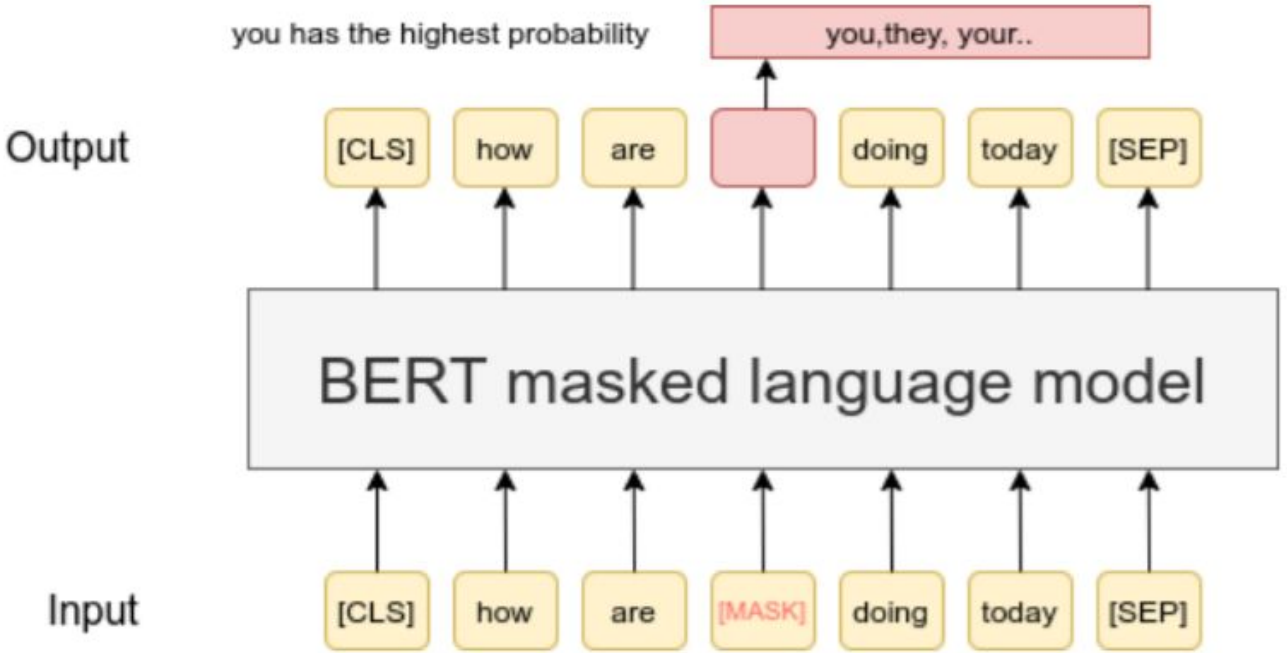
MLM: De forma aleatoria en cada inferencia se ocultó una palabra de cada secuencia (masked)

Utiliza la tokenización WordPiece

BERT (2019)

“Pre-training of Deep Bidirectional Transformers”

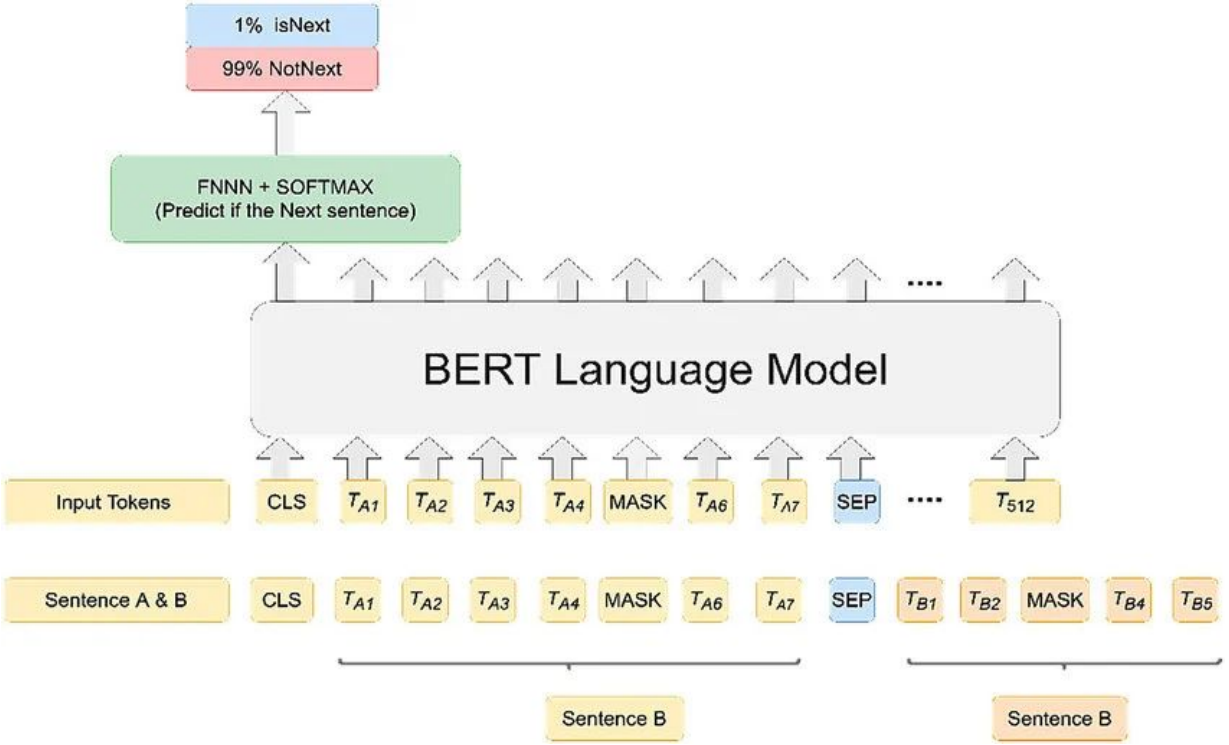
MLM



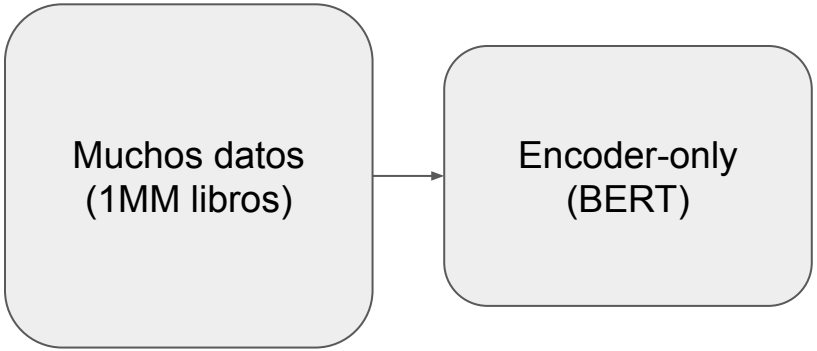
BERT (2019)

“Pre-training of Deep Bidirectional Transformers”

NSP



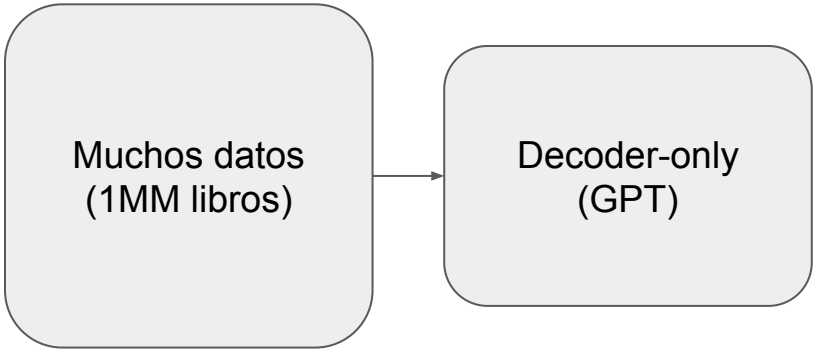
Pre-entrenamiento



Me siento feliiz gane la loteria.

Me siento [MASK] gane la loteria. Y = Feliz

Pre-entrenamiento



Generative pre-trained
transfomer

Conociendo a BETO (BERT español)

Link al Colab

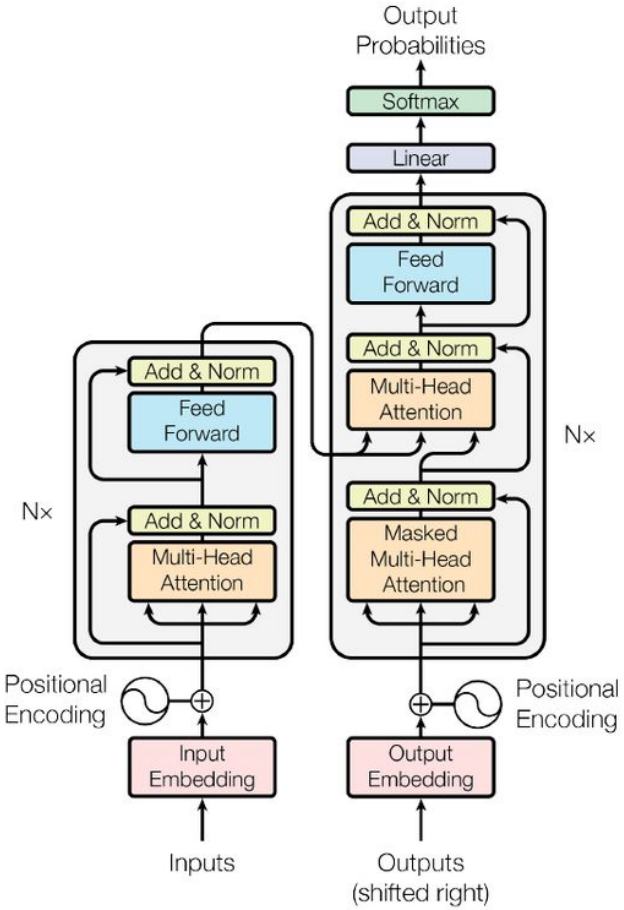
LINK

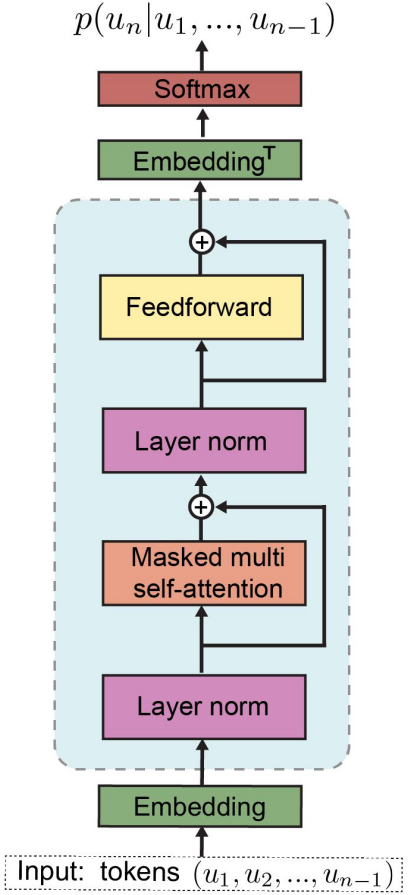
BERT - Sentiment analysis (IMDB dataset)

Link al Colab

LINK

GPT2


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

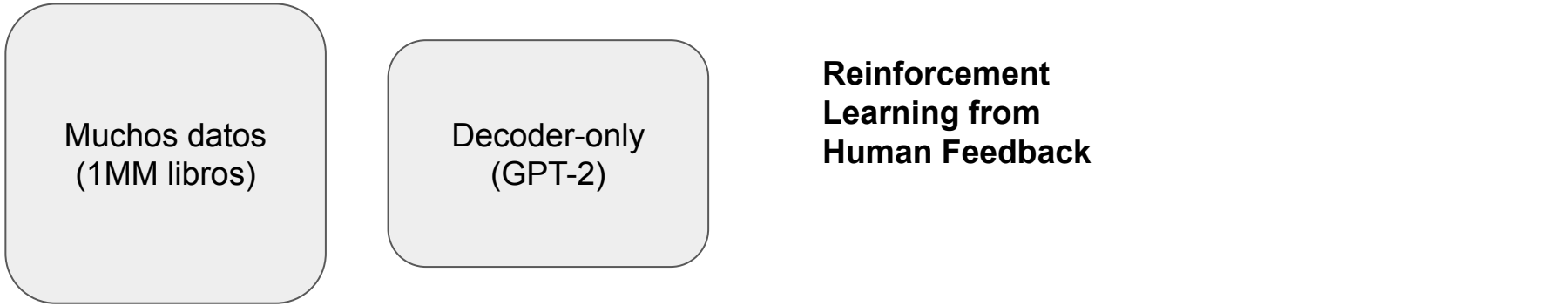

$p(u_n|u_1,...,u_{n-1})$
Softmax
Embedding^T
Feedforward
Layer norm
Masked multi self-attention
Layer norm
Embedding
Input: tokens $(u_1, u_2, ..., u_{n-1})$

GPT2

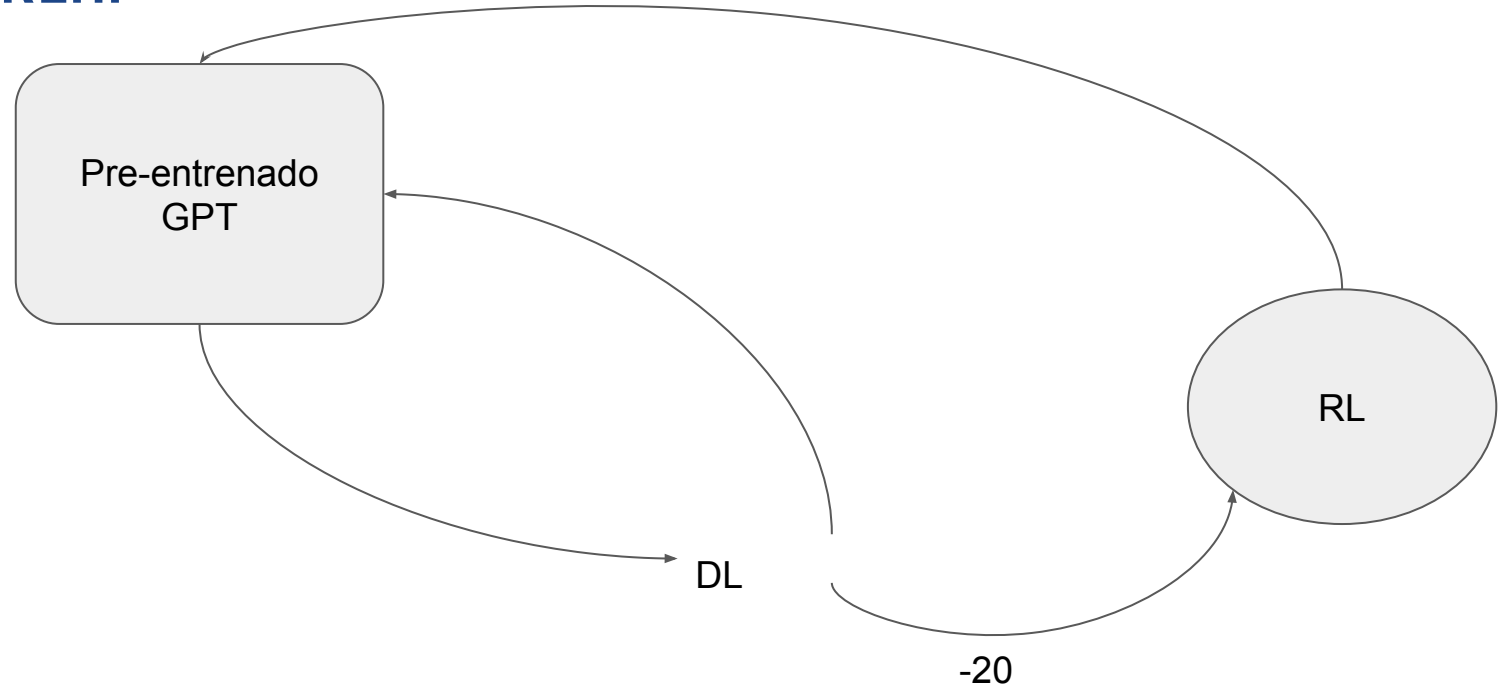
Link al Colab

LINK

Post-Training

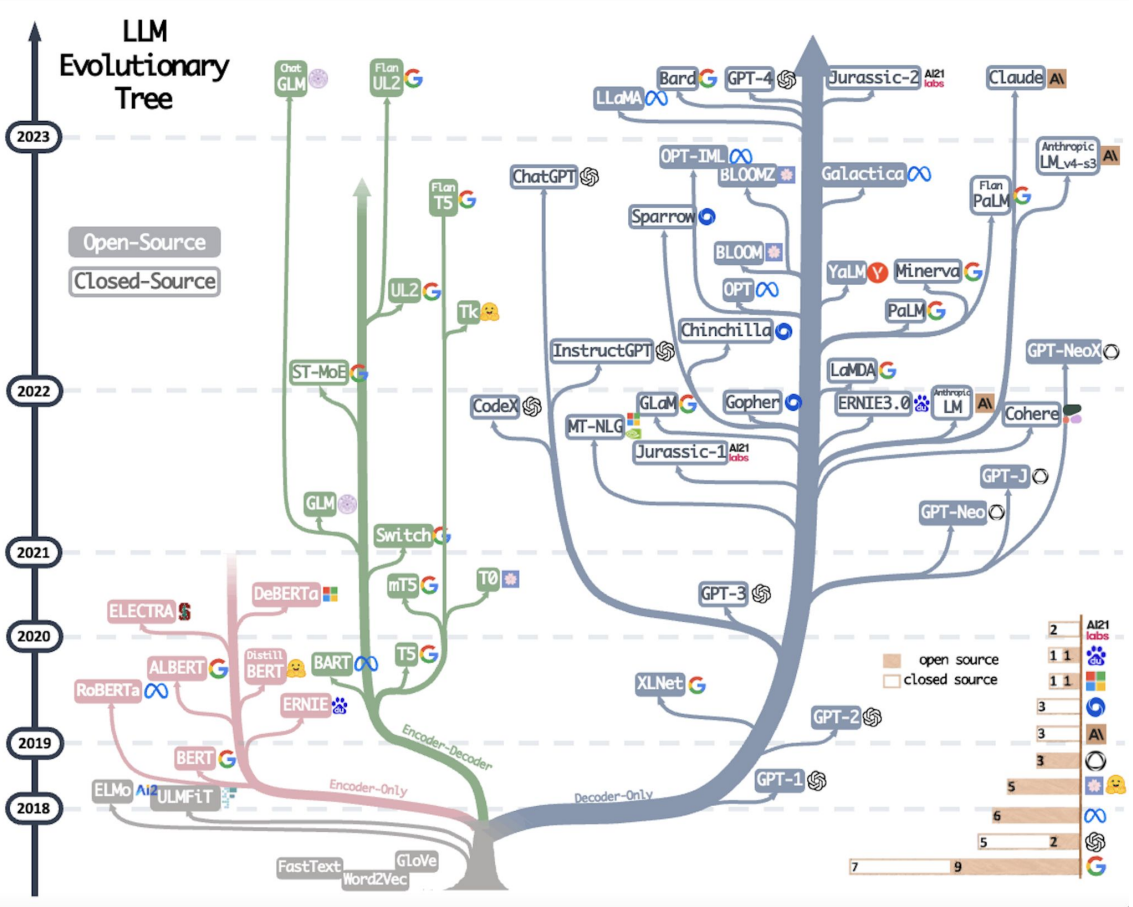


RLHF



2+2 - El resultado DE TU OPERACION es 4
Que es el iphone - Es un celular
…

LLMs


LLM Evolutionary Tree
2023
2022
2021
2020
2019
2018
Open-Source
Closed-Source
Chat GLM
Flan UL2
Bard
GPT-4
Jurassic-2
Claude
LLaMA
OPT-IML
BLOOMZ
Galactica
Anthropic LM_v4-s3
ChatGPT
Flan T5
Sparrow
Flan PaLM
BLOOM
YaLM
Minerva
UL2
OPT
Tk
PaLM
Chinchilla
InstructGPT
LaMDA
GPT-NeoX
ST-MoE
CodeX
GLaM
Gopher
ERNIE3.0
Anthropic LM
Cohere
MT-NLG
Jurassic-1
GPT-J
GLM
GPT-Neo
Switch
mT5
T0
GPT-3
ELECTRA
DeBERTa
ALBERT
Distill BERT
BART
T5
RoBERTa
ERNIE
XLNet
GPT-2
BERT
Encoder-Decoder
ELMo
ULMFiT
Encoder-Only
Decoder-Only
GPT-1
FastText
GloVe
WordZVec
open source
closed source
2
1 1
1 1
3
3
3
5
6
5 2
7 9

LLM Zoo

Link al Colab

LINK